

A Multi-cycle RV32I System on Chip for the Digilent Genesys 2

Final project report

Amit Malach

Supervisor: Prof. Avi Mendelson

Technion – Israel Institute of Technology

November 4, 2026

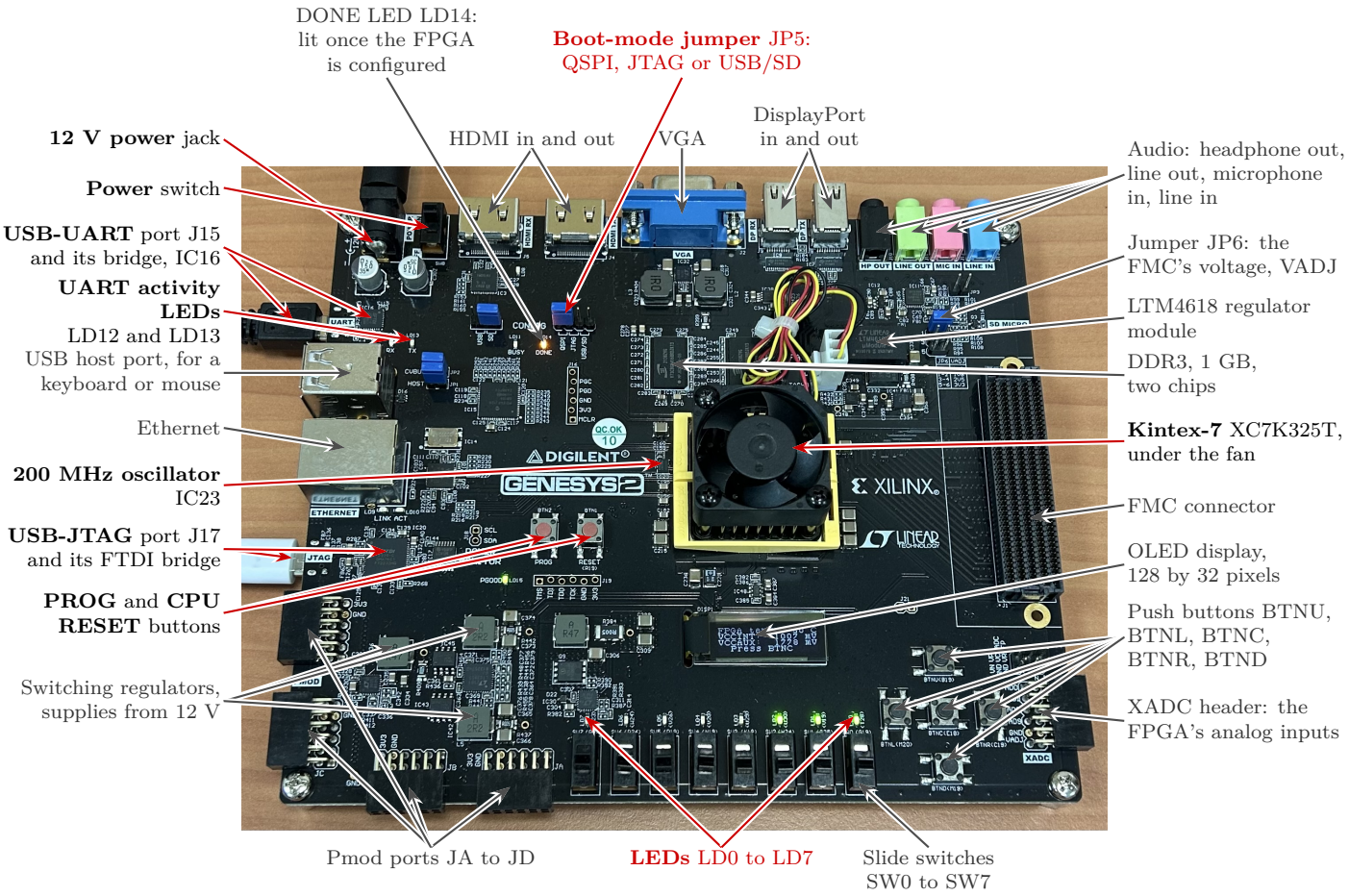


Figure 1: The Digilent Genesys 2 from above, powered on and running Digilent's factory demo. In **red**, the parts this design uses.

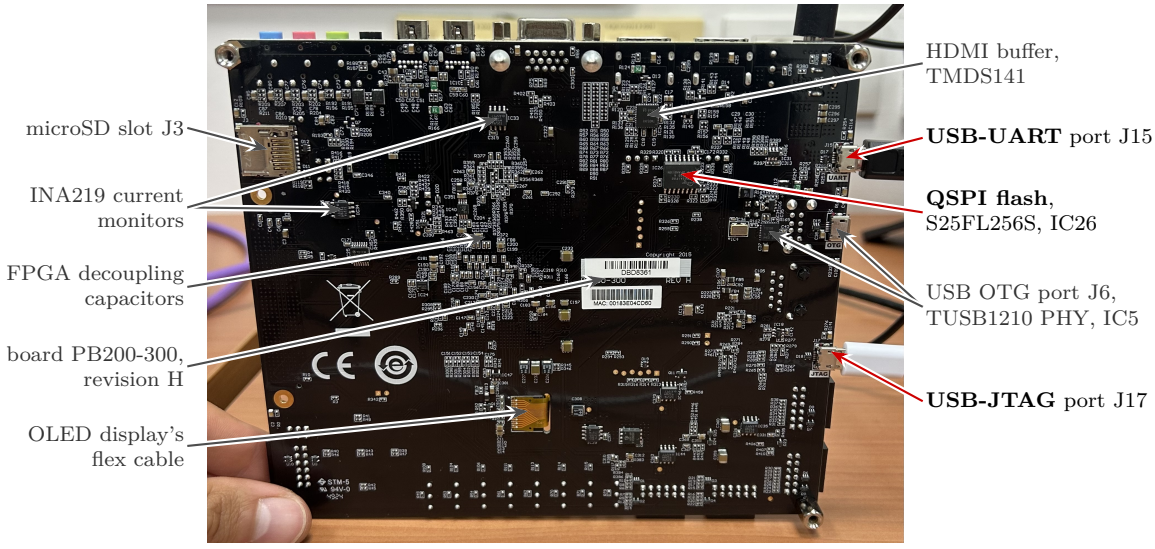


Figure 2: The underside, the top view mirrored. In **red**, the parts this design uses.

Abstract

This report describes a small system on chip built around a multi-cycle RV32I core, written in Verilog and run at 200 MHz on the Kintex-7 FPGA of a Digilent Genesys 2 board. The core executes the RV32I base integer instruction set, one instruction at a time, in four to six clock cycles each, under a seven-state finite state machine. Around it sit 6 KB of block RAM, a memory-mapped LED register, a UART transmitter and a controller for the board's 32 MB QSPI flash, all on one bus that the core sees as plain memory. Two boot modes share the same hardware. In BRAM boot the firmware image is part of the bitstream and the program runs from the RAM. In flash boot the core executes a program in place from the flash, which admits programs larger than the RAM at about a hundredth of the speed.

On the board the design draws the Mandelbrot set and a turning RISC-V logo over the UART from its block RAM, renders tinyraytracer's ray-traced picture, a program five times the RAM's size, in place from the flash, and plays the ST-NICCC polygon demo from scene data in the flash. The design is inspired by Bruno Levy's *From Blinker to RISC-V* series.

Contents

1 Introduction **1**
- 1.1 What the board does . . . 1
- 1.2 Starting point and own work . . . 3
- 1.3 What was built . . . 4
- 1.4 Results in brief . . . 4
- 1.5 How to read this report . . . 5

2 Board and tools **6**
- 2.1 Tools . . . 12

3 Repository tour **14**

4 System architecture **15**
- 4.1 The bus and the modules . . . 16
- 4.2 Memory map and address decode . . . 16
- 4.3 Clock and reset . . . 17

5 The RV32I instruction set **18**
- 5.1 Registers and assembly syntax . . . 18
- 5.2 Instruction formats . . . 18
- 5.3 The instructions . . . 19
- 5.4 The calling convention . . . 22

6 The multi-cycle core **24**
- 6.1 The state machine . . . 24
- 6.2 The datapath . . . 24
- 6.3 One instruction through the states . . . 26

7 Memory and memory-mapped IO **28**
- 7.1 The RAM . . . 28
- 7.2 The LED register and the UART . . . 28
- 7.3 The flash stall . . . 28

8 Boot modes and the SPI flash **31**
- 8.1 The flash layout . . . 31
- 8.2 CCLK and STARTUPE2 . . . 32
- 8.3 Execute-in-place . . . 32

9 From C to the board **33**

10 Verification **35**
- 10.1 The ALU unit bench . . . 35
- 10.2 The golden trace . . . 35
- 10.3 The regression . . . 36
- 10.4 The gates and the lint . . . 36

11 Implementation results **37**
11.1 Utilization 37
11.2 Timing 37
11.3 Power 39
11.4 The design on the device 40

12 For the next student **41**
12.1 Pitfalls 41
12.2 Extensions, ranked 43

13 Conclusions **44**

Bibliography **45**

1 Introduction

This is an undergraduate project with one aim: to build a processor that runs compiled C on a real FPGA board, and to understand every layer between the C source and the lit LED. The result is a multi-cycle RV32I core, 6 KB of RAM, three memory-mapped IO registers and a flash controller, wired into one top level and run on a Digilent Genesys 2 at the board's 200 MHz clock. This chapter shows what the board does with it, then what was built and how the report is laid out.

1.1 What the board does

Every picture in this section is rendered from bytes the board sent over its UART. From the moment a bitstream starts to load over JTAG, the host records the UART at 115 200 baud, 8N1, with a timestamp on every read, accurate to about 2 ms. The raw captures are committed in `docs/design/captures/`, and `make figures` renders the figures from them. The demos draw with ANSI escape sequences, each pixel a coloured terminal cell, so a serial terminal shows them as pictures. The QR code on the right of each figure holds the address of the repository's README, `https://github.com/amitmalach/FPGA_RiscV_MultiCycle#running-the-demos`, where every demo below plays as an animation.

Mandelbrot. The committed firmware image draws the Mandelbrot set from the block RAM, in fixed-point arithmetic with at most 20 iterations a pixel. It sends one frame of 46 by 46 pixels a second, steps the colours by one at each frame, and counts the frames on the LEDs. Figure 1.1 shows the first five of the 23 frames captured, which cover a full cycle of the 21 colours.

The first frame the board sent is byte for byte the frame the simulation of the design produced (the golden trace, Section 10.2) and the output of the same C program run on a PC, so the board, the simulation and the C source agree. It took 1031 ms on the board, and the simulation counted 206.3 million cycles for it, which take 1031 ms at 200 MHz: the board runs the cycles the simulation counts. About 956 ms of the frame is the UART sending its bytes, so the computation itself takes about 75 ms.

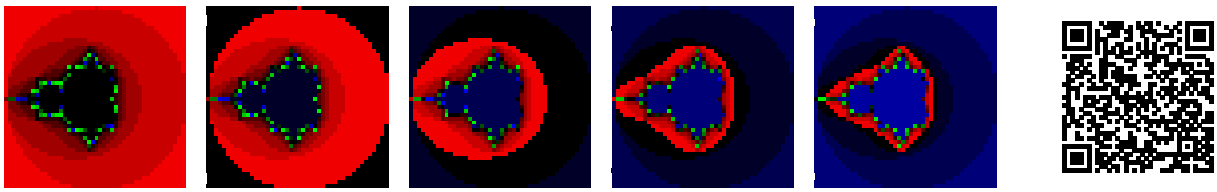


Figure 1.1: The mandelbrot demo on the board, in BRAM boot: the first five of the 23 frames captured, left to right, about a second apart, each pixel two terminal cells on a 24-bit background colour. The animation plays in the repository's README, at `https://github.com/amitmalach/FPGA_RiscV_MultiCycle#running-the-demos`, the address the QR code on the right holds.

riscv_logo. The riscv_logo demo rotates and zooms the RISC-V logo, a 16 by 16 bitmap, over a grid of 40 by 40 pixels in the 16 standard terminal colours. One turn is 64 frames, about half a second each, a pace the UART sets. Figure 1.2 shows every eighth frame. The captured frames are, byte for byte, what `fw/riscv_logo.c` prints when compiled natively.

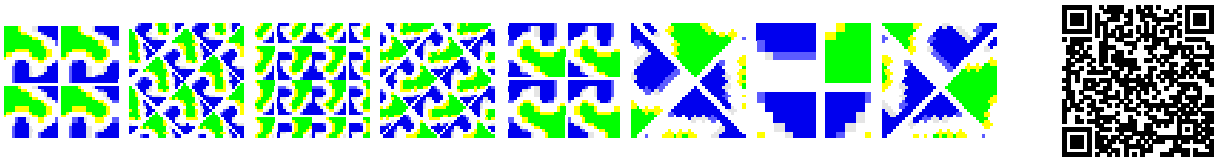


Figure 1.2: The riscv_logo demo on the board, in BRAM boot: frames 0, 8, ..., 56 of one turn, left to right, 45 degrees apart. The whole turn plays in the README, at the address the QR code on the right holds.

tinyraytracer. tinyraytracer is Dmitry Sokolov's small ray tracer [5], a few hundred lines written to teach how a ray-traced picture is made. For each pixel it casts a ray from the eye through the screen, finds the nearest sphere or floor point the ray hits, and colours it from three point lights, with diffuse and specular shading and shadows. Rays that hit the mirror or the glass sphere are reflected or refracted and traced again. Here it prints its picture to the terminal as ANSI colour cells. It is 29 KB of code and constants, five times the RAM, so it runs in flash boot: every instruction is fetched in place from the board's flash, and every floating-point operation is a call into `libgcc`. The board drew the picture row by row in about two minutes, and Figure 1.3 shows it as it grew. Its output is, byte for byte, what `fw/tinyraytracer.c` prints when compiled natively. Two board problems showed on the way: the board printed nothing until the CPU RESET button was pressed after the load (pitfall P5), and writing the image to the flash cost the board its power-on boot until the flash's Quad Enable bit was set again (pitfall P8).

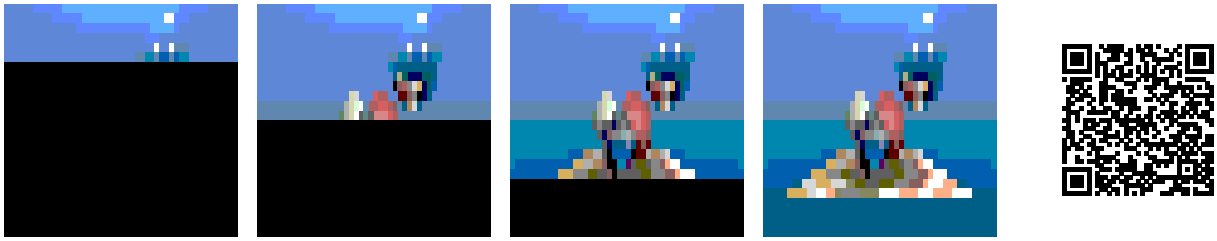


Figure 1.3: tinyraytracer on the board, in flash boot, after 6, 12, 18 and all 24 rows of its picture: 24, 64, 107 and 127 s after its first pixel. Four spheres, of ivory, glass, red rubber and mirror, over a chequered floor and lit by three lights, 48 by 24 terminal cells in xterm's 256 colours. The README plays the drawing, 20 times faster than the board drew it.

sieve. The sieve of Eratosthenes finds the primes below a bound by crossing out the multiples of each prime in turn: whatever is never crossed out is prime. The sieve demo, taken from the PicoRV32 project, sieves the odd numbers from 3 to 129 in a 64-bit bitmap, prints the 31 primes below 128, one a line, and then a checksum of them, `1772A48F`, followed by `OK` when the checksum is the expected one, and starts again after a short delay, a round every 96 ms. Its output is text, so Figure 1.4 shows it as a terminal 12 rows high shows it: once the screen is full, each new line scrolls it up. The captured rounds are, byte for byte, what `fw/sieve.c` prints when compiled natively, but for the checksum's hex digits, which `fw/print.c` prints in upper case.

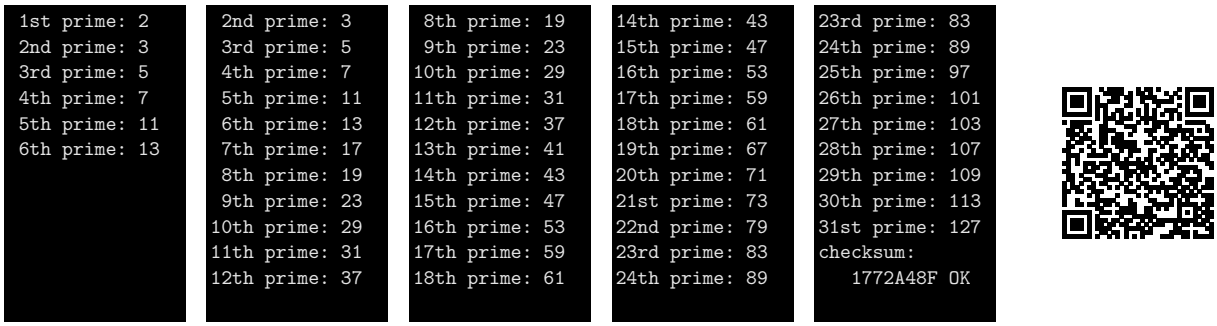


Figure 1.4: The sieve demo on the board, in BRAM boot: a terminal 12 rows high after 6, 12, 18, 24 and 33 lines of the first round, 7, 15, 23, 30 and 42 ms after its first byte. The last line of each screen is where the cursor waits. The README plays the first two rounds, 50 times slower than the board sent them.

ST_NICCC. ST-NICCC 2000 is a demo for the Atari ST by the group Oxygene whose best-known part is a flight through a scene of flat-shaded polygons, streamed from disk. Its scene file was later published for a competition to replay it on other machines [6]. Each frame of the file lists polygons, each with a colour from a palette of 16 and the screen points of its corners. The player, `fw/ST_NICCC.c`, reads the file through the core's flash region and draws each frame as 128 by 50 coloured cells. The file is 640 KB, a hundred times the RAM, so it was written to the flash at `0xC00000`, where Figure 8.1 reserves room for it, and the player runs from the block RAM in BRAM boot. The board drew a frame every 0.9 s, with the UART sending at its full rate. The player never clears the screen: each frame is drawn over the last, so a screen taken in the middle of a frame shows parts of two.

The run is also the clearest evidence behind pitfall P5. Its first frame came out wrong, as a wrong first word from the flash makes it: the first SPI transaction after configuration loses clock edges (Section 8.2). The polygons were right again within two seconds, and one palette colour stayed wrong until the scene set it again, 70 s in. From there on, the output is byte for byte what `fw/ST_NICCC.c` prints when compiled natively with the same scene file.

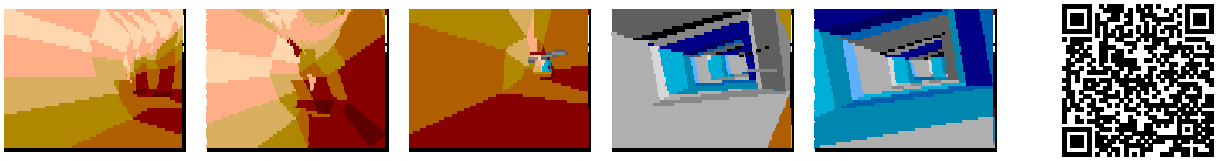


Figure 1.5: ST_NICCC on the board, in BRAM boot, with its scene data in the flash: the screen 76, 97, 118, 139 and 145 s after the player's first byte, 128 by 50 terminal cells in xterm's 256 colours. The README plays the screen every half second, in real time, from 70 s to the end of the capture.

1.2 Starting point and own work

The project started from Bruno Levy's *learn-fpga* series *From Blinker to RISC-V* [1], which grows a multi-cycle RV32I core in small increments, through memory-mapped IO and the GNU toolchain to execution from SPI flash, for small boards such as the IceStick and the ULX3S. The series supplied the starting point: the shape of the state machine, the bus and the demo programs. Taking that starting point to a working, verified system on the Genesys 2, a board the series does not support, was the project's own work, and most of the effort went there.

- **The port to the Genesys 2.** The differential 200 MHz clock, the pin constraints, the board's UART pin names (pitfall P1), and the flash's clock, which a 7-series device reaches

only through `STARTUPE2` (Section 8.2).

- **The RTL, restructured and debugged.** Eleven modules, one per file, with the memory map defined once and every copy of it checked. The bugs found and fixed include the arithmetic shift that shifted logically (pitfall P4) and a simulation hang in the flash controller (pitfall P3), and the design notes record the reason behind every subtle choice, such as the reset state (pitfall P6).
- **The verification.** The golden trace and the regression, the ALU unit bench, ten lint rules, and the hooks that hold every commit to its gates (Chapter 10).
- **The board work.** The build flow from C to the board in both boot modes, the scripts that capture every demo's output and check it against the PC, and a small design of its own that read and repaired the flash's Quad Enable bit (pitfall P8).
- **The analysis.** The timing, utilisation and power of the routed design (Chapter 11), and this report, whose figures are generated by scripts from the real build and the real board.

1.3 What was built

The hardware is eleven Verilog modules, one per file, in one clock domain (Chapter 4). The core sees a single bus, a 32-bit address with a read strobe, write data with a byte mask and a busy input. `IOBus` decodes it into three regions by one address bit each: the RAM, the IO registers and the flash. The RAM is 6 KB of block RAM shared by instruction fetch and data. The IO registers are the LEDs, the UART's data register and its status word, and the flash controller turns a read in flash space into one SPI transaction and holds the core busy for its length.

The core (Chapter 6) executes RV32I and nothing beyond it: no multiplier, no interrupts, no control and status registers. An instruction takes four cycles, a store five and a load six, each cycle a memory or register-file read answered one cycle after it is asked. The same hardware boots two ways (Chapter 8): in BRAM boot the firmware image is read into the block RAM when the bitstream is synthesised. In flash boot the reset vector points into the flash and the core fetches each instruction from there in place.

The firmware (Chapter 9) is six demos in C and assembly, built by the RISC-V GCC with a start-up file and a linker script per boot mode. The repository (Chapter 3) holds the RTL, the firmware, the simulation, the Vivado scripts and this report, with make targets that build and check all of it. Verification (Chapter 10) is an ALU unit bench, a golden trace that the regression must reproduce byte for byte and cycle for cycle, and a lint. Each is a gate whose log a commit cites.

1.4 Results in brief

- **Size and speed.** The routed design is 984 LUTs, 244 flip-flops and two block RAM tiles, under half a percent of the XC7K325T. The 200 MHz clock is met with 0.219 ns of slack on the worst path, the load path (Chapter 11).
- **The board, the simulation and the source agree.** The first mandelbrot frame the board sent is byte for byte the golden trace's UART output, which is byte for byte what the C program prints when compiled natively. The board's frame time matches the simulated cycle count at 200 MHz (Section 1.1).
- **Flash boot works**, with one press of the reset button after each load, which pitfall P5 explains.

- **The demos are bound by the UART.** At 115 200 baud a byte takes 87 µs, and the mandelbrot frame spends about 956 of its 1031 ms sending. A faster core would not show on these demos.
- **Ten pitfalls are recorded**, P1 to P10 in Chapter 12, each with its symptom, its cause and where it is now handled. The costliest were outside the core.

1.5 How to read this report

Chapters 2 to 9 describe the system from the outside in: the board and tools, the repository, the architecture, the RV32I instruction set, the core, the memory and IO, the boot modes, and the path from a C file to the board. Chapters 10 and 11 are the evidence beside Section 1.1: how the design is checked and what Vivado made of it. Chapter 12 is for whoever extends the design, and Chapter 13 concludes.

The design notes, `docs/design/notes.md`, hold the full reasoning behind each module. The report states each choice and cites the notes' section by its heading, as (`notes.md`, "section"). Every figure that a script can make is regenerated by `make figures` in `docs/design/`, and the recipe for each manual one is in the figures' README.

2 Board and tools

The Digilent Genesys 2 (Figures 1 and 2) carries a Kintex-7 XC7K325T, speed grade −2, in the 900-ball FFG900 package, with 1 GB of DDR3, HDMI and DisplayPort, Ethernet, audio, an FMC connector and five Pmods [3]. The design uses seven of its parts (Table 2.1). Five are wired to the design itself: the oscillator, the reset button, the LEDs, the USB-UART bridge and the flash connect to the nine ports of `SOC`, 16 FPGA pins in all. The other two, the USB-JTAG bridge and the mode jumper, connect to no port: they serve only to load a bitstream into the FPGA. Everything else is room to grow (Chapter 12).

Part (schematic page)	FPGA board pin names	Design port
200 MHz LVDS oscillator, DSC1103 (IC23, p. 13)	AD12, AD11	`sysclk_p`, `sysclk_n`
CPU RESET button (BTN1, p. 11)	R19	`RESET`, low while pressed
LEDs LD0 to LD7 (p. 4)	T28, V19, U30, U29, V20, V26, W24, W23	`LEDS[0]` to `LEDS[7]`, high to light
USB-UART bridge, FT232RQ (IC16, p. 9), on the micro-USB port J15	Y23, Y20	`uart_rx_out`, `uart_tx_in`
QSPI flash, S25FL256S, 32 MB (IC26, p. 15)	U19, P24, R25, B10	`spiflash_csn`, `spiflash_mosi`, `spiflash_miso`, and the clock through `STARTUPE2`
USB-JTAG bridge, FT2232HQ (read from its marking: the schematic draws only its JTAG nets, p. 15), on the micro-USB port J17	F10, E10, H10, G10 (TMS, TCK, TDI, TDO)	none: it loads bitstreams
Mode jumper JP5 (p. 15)	AB1, AB2, AB5 (M2, M1, M0)	none: it picks what configures the FPGA at power-on

Table 2.1: The Genesys 2 parts the design uses. The pin names are the FPGA package's ball names, as the FPGA symbol of Digilent's schematic, revision H.1 [4], labels them, in the order of the ports beside them. Parts and pages are the schematic's, and the pins of the design's ports are also those of `constraints/Genesys-2-Master.xdc`.

Figure 2.1 shows the five connected parts and the mode jumper in Digilent's schematic [4], and each paragraph below reads the panel that follows it.

The oscillator (Figure 2.1a). IC23, a DSC1103, is the board's clock: a chip with a tiny resonator inside that outputs a square wave, here 200 million cycles a second, for as long as it has power. It takes the 3.3 V supply, VCC3V3, on its VDD pin, and its enable pin, EN, is tied to the same supply, so it never stops.

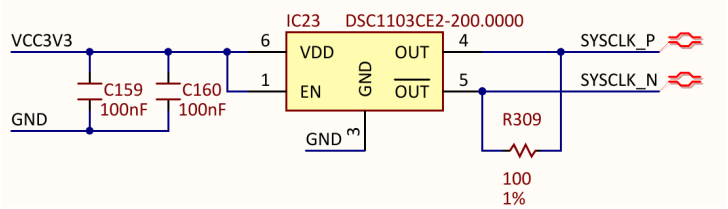
Why the two capacitors. C159 and C160, 100 nF each, connect the supply to ground right beside the chip. They are *decoupling* capacitors, and yes, their purpose is to keep supply noise away from the clock. At every edge of its output the oscillator draws a short spike of current. The supply reaches it through long copper traces, which resist such fast changes, so on its own the voltage at VDD would dip a little at every edge, and a noisy supply shows up as *jitter*: clock edges that come slightly early or late. A capacitor is a small store of charge. Placed at the pin, it delivers each spike itself and is refilled slowly from the supply, and it also passes fast noise

that arrives on the supply line straight to ground. Two in parallel store twice the charge and pass noise to ground more easily than one.

Why two outputs, one the complement of the other. OUT, pin 4, drives the net `SYSCLK_P`, and $\overline{\text{OUT}}$, pin 5, the inverted output, drives `SYSCLK_N`: whenever one is high the other is low. This is *differential* signalling. The receiver does not ask whether one wire is high, it asks which of the two is higher, and that pays three ways. Noise picked up on the way, from a neighbouring signal or from a ground that is not exactly 0 V, lands on both wires almost equally, since they run side by side, and cancels in the difference. The two wires carry equal and opposite currents, so the pair itself radiates little noise. And since only the sign of the difference matters, a swing of a few hundred millivolts is enough, which switches faster and with less power than a full swing from 0 to 3.3 V. Every edge of this 200 MHz clock times the whole design, so a clean edge is worth the second wire. One wire would carry the same clock, with more of the noise.

What an LVDS pair is. LVDS, low-voltage differential signalling, is a common standard for such pairs, and the one the constraints file sets for these two pins (`IOSTANDARD LVDS`). The driver pushes a constant current of about 3.5 mA out along one wire and back along the other, and the bit it sends sets the direction. At the far end the current passes through a 100 Ω resistor across the two wires, which turns it into about ±350 mV, and the receiver reads only the sign of that voltage.

What R309's termination means. At 200 MHz an edge does not reach the whole wire at once: it travels along the pair of copper traces as a wave, and the pair behaves as a *transmission line* with a *characteristic impedance*, 100 Ω for a pair laid out for LVDS. Wherever the wave meets a different impedance, part of it is reflected back, like an echo. An FPGA input on its own is almost an open circuit, so without anything at the end the edge would bounce back and forth along the pair and *ring*, adding wiggles that the receiver could take for extra edges. A resistor across the end of the line equal to its impedance absorbs the wave entirely, and nothing comes back: that is what *terminating* a line means. R309, 100 Ω to within 1%, is this pair's termination, and also the resistor the LVDS current turns into a voltage across. The constraints file does not turn on the FPGA's own internal termination (`DIFF_TERM`), so R309 is the only one. The pair reaches the FPGA on AD12 and AD11, a pin pair that can drive the FPGA's clock network directly, and enters the design through `IBUFDS`, the differential input buffer (Section 4.3).

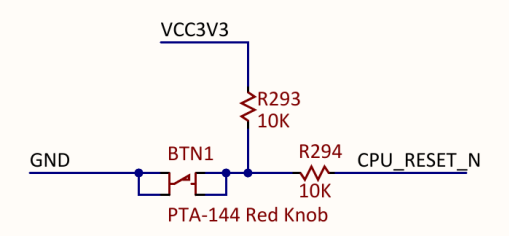


(a) The oscillator, IC23 (p. 13): an LVDS pair terminated by R309.

Figure 2.1: The parts of Table 2.1 in Digilent's Genesys 2 schematic, revision H.1 [4], cropped by `docs/design/scripts/schematic_crops.sh`: panels (a) to (f), each under the paragraph that reads it.

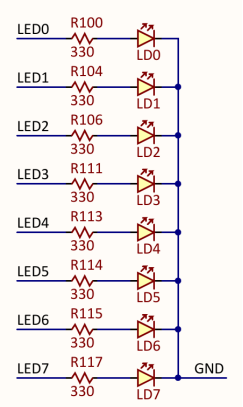
The reset button (Figure 2.1b). BTN1 is a push button: while it is pressed, it connects its two sides, here a node and ground. A pin wired only to an open button would *float* while the button is up: nothing would set its level, and it would read 0 or 1 at random as it picked up noise. R293, 10 kΩ from the node to 3.3 V, is a *pull-up*: it holds the node at 3.3 V, a 1, while the button is up. A press connects the node straight to ground, which overrides the resistor, and the node reads 0, while the resistor passes a harmless 0.33 mA, 3.3 V over 10 kΩ. R294, 10 kΩ in series, carries the node to the net `CPU_RESET_N` on FPGA pin R19. An input draws

almost no current, so R294 changes no level, but it protects the pin: should a design ever drive R19 high as an output while the button shorts the node to ground, R294 limits the current to 0.33 mA instead of a short circuit. The net is low only while the button is pressed, as the `_N` in its name says, hence the design's active-low reset. A mechanical contact also *bounces*, opening and closing for a few milliseconds before it settles. The design does not filter that out: a reset that repeats a few times only restarts the design a few times.



(b) CPU RESET, BTN1 (p. 11): pulled up, pressed to ground.

The LEDs (Figure 2.1c). Eight identical branches, one per LED. An LED, a light-emitting diode, passes current one way only, in the direction its triangle points, and gives light while it does; the two small arrows of its symbol are the light. It starts to conduct only once the voltage across it passes its forward voltage, about 2 V for a typical LED, and above that its current rises so steeply that something else must set it: the resistor. Each net, `LED0` to `LED7`, comes from an FPGA pin, passes a 330 Ω resistor and the LED, and ends on the common ground at the right. With the pin high, at 3.3 V, about 1.3 V is left across the resistor, so about 4 mA flows: enough to light the LED, and well within what an FPGA pin can drive. With the pin low there is no voltage across the branch and the LED is dark. So an LED lights while its pin is high. The pin follows one bit of the design's LED register (Section 4.2): a store that writes 1 to bit i lights LDi, and 0 puts it out.

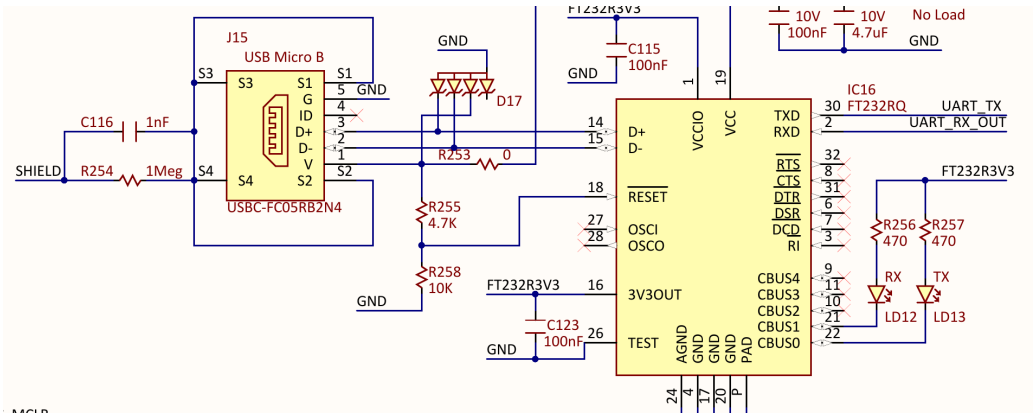


(c) LEDs, p. 4.

The USB-UART bridge (Figure 2.1d). A UART sends bytes over one wire, one bit at a time at an agreed rate, here 115 200 bits a second, each byte framed by a start bit and a stop bit. A PC has no UART pins, only USB, so the board carries a *bridge*: IC16, an FT232RQ, shows up on the PC as a USB serial port (`/dev/ttyUSB0` on Linux), and on its other side drives and reads UART lines. From left to right:

- *J15*, the micro-USB connector. V carries the PC's 5 V supply, D+ and D− carry USB's data, itself a differential pair, as the clock is, and G is ground. ID serves only USB On-The-Go and is left unconnected, and S1 to S4 are the connector's metal shell.

- *The shell.* R254, 1 MΩ, and C116, 1 nF, connect the shell to the net `SHIELD`. The capacitor passes fast noise picked up by the cable's shield, and the large resistor slowly drains static charge, yet neither joins the PC's ground to the board's a second time through the shield, which would let current loop through the cable.
- *D17*, a diode array between the data lines and ground. A finger touching the connector can discharge thousands of volts of static electricity into it. The diodes conduct as soon as a line goes far outside its normal range, and send the pulse to ground before it reaches IC16. In normal use they do nothing.
- *The divider on* $\overline{\text{RESET}}$. R255, 4.7 kΩ, and R258, 10 kΩ, divide the PC's 5 V down to about 3.4 V, a 1, on the bridge's active-low reset input. Without a powered cable V is at 0 V and the bridge is held in reset, so it runs only while a PC powers the cable.
- *The bridge's own supply.* VCCIO, which sets the voltage of the bridge's signal pins, comes from the bridge's own 3.3 V output, 3V3OUT, the net `FT232R3V3`, so its UART lines switch between 0 and 3.3 V, the levels of the FPGA's bank. C115 and C123, 100 nF each, decouple those supplies, as at the oscillator. TEST must be grounded in normal use, and OSCI and OSCO, for an outside crystal, stay unconnected because the chip keeps time on its own.
- *The UART lines*, on the right. TXD, the bridge's output, drives `UART_TX`, which carries the PC's bytes toward the FPGA. RXD, its input, receives `UART_RX_OUT`, the FPGA's output and the only line this design drives. The board's net names take the PC's view for TX and RX and the FPGA's for in and out, and mixing the two up is pitfall P1 (`notes.md`, "UART port names and the pin-direction trap").
- *The activity LEDs.* CBUS0 and CBUS1 drive LD13 (TX) and LD12 (RX). Each LED hangs from `FT232R3V3` through 470 Ω into its CBUS pin, so it lights while the bridge pulls that pin low, which it does in pulses while data passes [7]. The bridge names them from its own view as well: data it receives on RXD blinks RX, so LD12 is the one that blinks while this design sends.
- *The modem lines*, RTS to RI, are the handshake wires of the old RS-232 serial standard. Nothing here uses them, and the crosses mark them unconnected.



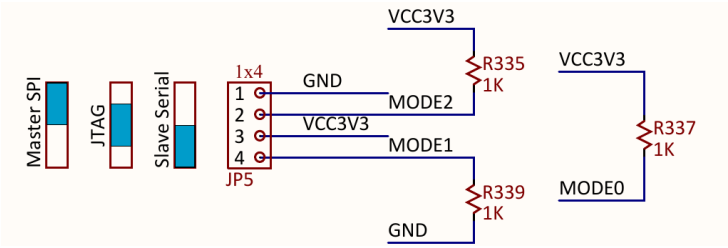
(d) The USB-UART bridge, IC16, behind the micro-USB port J15 (p. 9).

The mode jumper (Figure 2.1e). An FPGA forgets its design when the power goes: the bitstream is held in memory cells that need power, so at every power-on something must load it again. Three *mode pins*, M2 to M0, tell the FPGA where from, and the jumper sets them. JP5 is a row of four pins, and a *jumper* is a small removable cap that connects two neighbouring pins. The four are ground, the net `MODE2`, the 3.3 V supply and the net `MODE1`. Each mode net has a 1 kΩ resistor that gives it a default level: R335 pulls `MODE2` up, R339 pulls `MODE1` down,

and R337 holds `MODE0` high for good. The jumper connects a net straight to ground or to 3.3 V, which overrides its resistor. The three drawings on the left show the cap's positions:

- *Pins 1 and 2*, Master SPI, $M[2:0] = 001$: the cap grounds `MODE2`. The FPGA drives the flash's clock itself and reads its bitstream from the flash. On the owner's board the flash holds Digilent's factory demo at offset 0, left untouched.
- *Pins 2 and 3*, JTAG, $M[2:0] = 101$: the cap ties `MODE2` to 3.3 V, where its pull-up already holds it, so this is also the mode with no jumper at all. The FPGA loads nothing and waits for a bitstream over JTAG.
- *Pins 3 and 4*, Slave Serial, $M[2:0] = 111$: the cap raises `MODE1`. Another chip supplies the clock and the bitstream: the board's microcontroller, which configures the FPGA from a file on a USB stick or a microSD card [8].

The FPGA reads the mode pins when configuration starts, at power-on or after a press of PROG. JTAG works in every mode: the second micro-USB port is Digilent's USB-JTAG circuit, and Vivado's hardware manager loads every bitstream of this report through it, and writes the flash through it, whatever the jumper says.



(e) The mode jumper, JP5 (p. 15), and its three positions.

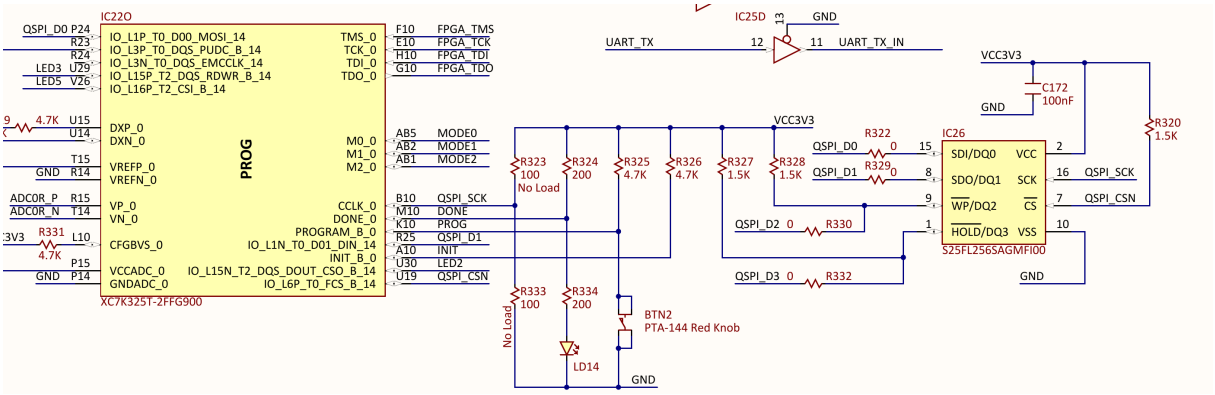
The configuration bank and the flash (Figure 2.1f). A chip with 900 balls is drawn in parts: on the left is IC22O, the part of the FPGA's symbol that holds bank 0, its dedicated configuration pins (labelled PROG), with a few pins of bank 14. Its right side, top to bottom:

- *TMS, TCK, TDI and TDO*, the JTAG port: a four-wire serial interface for testing and programming chips. TCK is its clock, TMS steers it from one step of its protocol to the next, TDI carries data in and TDO data out. Vivado reaches it through the USB-JTAG bridge.
- *M0 to M2*, the mode pins, from the jumper.
- *`CCLK_0`* on B10, the configuration clock. In Master SPI mode the FPGA drives the flash's clock, `QSPI_SCK`, on it. After configuration a design can drive it only through `STARTUPE2` (Section 8.2).
- *`DONE`* goes high once configuration has ended. The FPGA can only pull it low, so R324 pulls it up once the FPGA lets go, and it lights LD14 through R334, the board's "configured" LED.
- *`PROGRAM_B`*, active low, clears the FPGA and starts its configuration again. R325 pulls it up, and BTN2, the PROG button, grounds it.
- *`INIT_B`*, pulled up by R326: the FPGA holds it low while it clears its configuration memory, and pulls it low again if the bitstream it reads is corrupt.

R323 and R333, marked *No Load*, are places on the board for optional parts that are not fitted. On the left, `CFGBVS_0`, tied high through R331, tells the FPGA that its configuration banks run at 3.3 V, as the constraints file's `CFGBVS` and `CONFIG_VOLTAGE` say too. The bank 14 pins

at the bottom right are *dual-purpose*: during configuration they serve the flash, and afterwards they are ordinary pins for the design. P24, `D00_MOSI`, is `QSPI_D0`, R25, `D01_DIN`, is `QSPI_D1`, and U19, `FCS_B`, is `QSPI_CSN`, the pins the design drives as `spiflash_mosi`, `spiflash_miso` and `spiflash_csn`. Three of the LEDs are on the same bank.

On the right is IC26, the S25FL256S, a 32 MB flash memory, which keeps its contents without power. It speaks SPI, a serial interface of four wires: chip select, `CS`, active low, the flash listening only while it is low; a clock, `SCK`; and one data line each way, `SDI/DQ0` into the flash, which the FPGA drives as MOSI (master out, slave in), and `SDO/DQ1` out of it, MISO. In quad mode, QSPI, `WP/DQ2` and `HOLD/DQ3` carry data too, four bits a clock, but this design uses one bit each way. Each data line passes a 0 Ω resistor, R322, R329, R330 and R332: a plain link that the maker can remove to cut the line. R320 holds chip select high, the flash deselected, while nothing drives it, as before the FPGA is configured. R328 and R327 pull `WP/DQ2` and `HOLD/DQ3` high: as plain pins they are active low, write-protect and pause, and the design leaves them undriven, so the flash is neither write-protected nor paused. C172 decouples the flash's supply, as at the oscillator. The flash sits on the configuration pins so that the FPGA can read its bitstream from it at power-on, and its clock is the dedicated CCLK pin, which a design reaches only through `STARTUPE2` (Section 8.2). At the top, IC25D, a buffer that passes a level through unchanged, and always enabled since its enable input, pin 13, is grounded, carries the bridge's `UART_TX` to the FPGA as `UART_TX_IN`.



(f) The FPGA's configuration bank, PROG, and the QSPI flash, IC26 (p. 15). The flash's clock, `QSPI_SCK`, is the FPGA's CCLK pin, B10. At the top, the buffer IC25D carries the bridge's `UART_TX` to the FPGA as `UART_TX_IN`.

The package (Figure 2.2). The FPGA's silicon die sits inside a *package*, the square black part on the board, which connects it to the board's copper. The FFG900 is a *ball grid array*: instead of legs along its edges it has 900 small solder balls on its underside, a grid of 30 by 30 at 1 mm spacing, each soldered to a pad on the board. A ball is named like a square of a chessboard, by its row letter and its column number: AD12 is row AD, column 12. The rows run from A to AK, with I, O, Q, S, X and Z left out so that no letter can be read as a digit. Not every ball is a signal. Many are supply and ground, which share the chip's current between them and keep its supply steady, and others serve configuration, the transceivers or the on-chip analog-to-digital converter. Of the 900, 500 are *user I/O*, pins a design may use as it likes.

User I/O come in *banks*. A bank is 50 pins that share one output supply, VCCO, so all the pins of a bank work at one voltage: up to 3.3 V in the high-range (HR) banks, 12 to 18, and up to 1.8 V in the high-performance (HP) banks, 32 to 34 [9]. A GTX quad is four multi-gigabit transceivers, each with a transmit and a receive pair. Bank 0 holds the dedicated configuration pins, and the middle of the package the supplies of the FPGA's core.

To read the figure: it is Vivado's Package view of the routed design, the package seen from above, with the column numbers across the top and the row letters down the left. Each ball is

drawn as a symbol, and the key under the picture says what each symbol means: a grey circle is a user I/O pin, a pale hexagon a clock-capable one, a red mark a supply and a green mark ground. Each coloured outline groups the balls of one bank, and the labels around the picture name the banks and say what each serves on the Genesys 2. The design's 16 ports use 16 of the 500 user I/O, about 3%; they are the balls drawn in their modules' colours, and each module's box points at its pins. They sit in three banks: the LEDs, the reset button and the flash on bank 14, the UART on bank 12 and the clock on bank 33. The flash's clock has no user pin: it leaves on CCLK, a configuration pin of bank 0, through `STARTUPE2`.

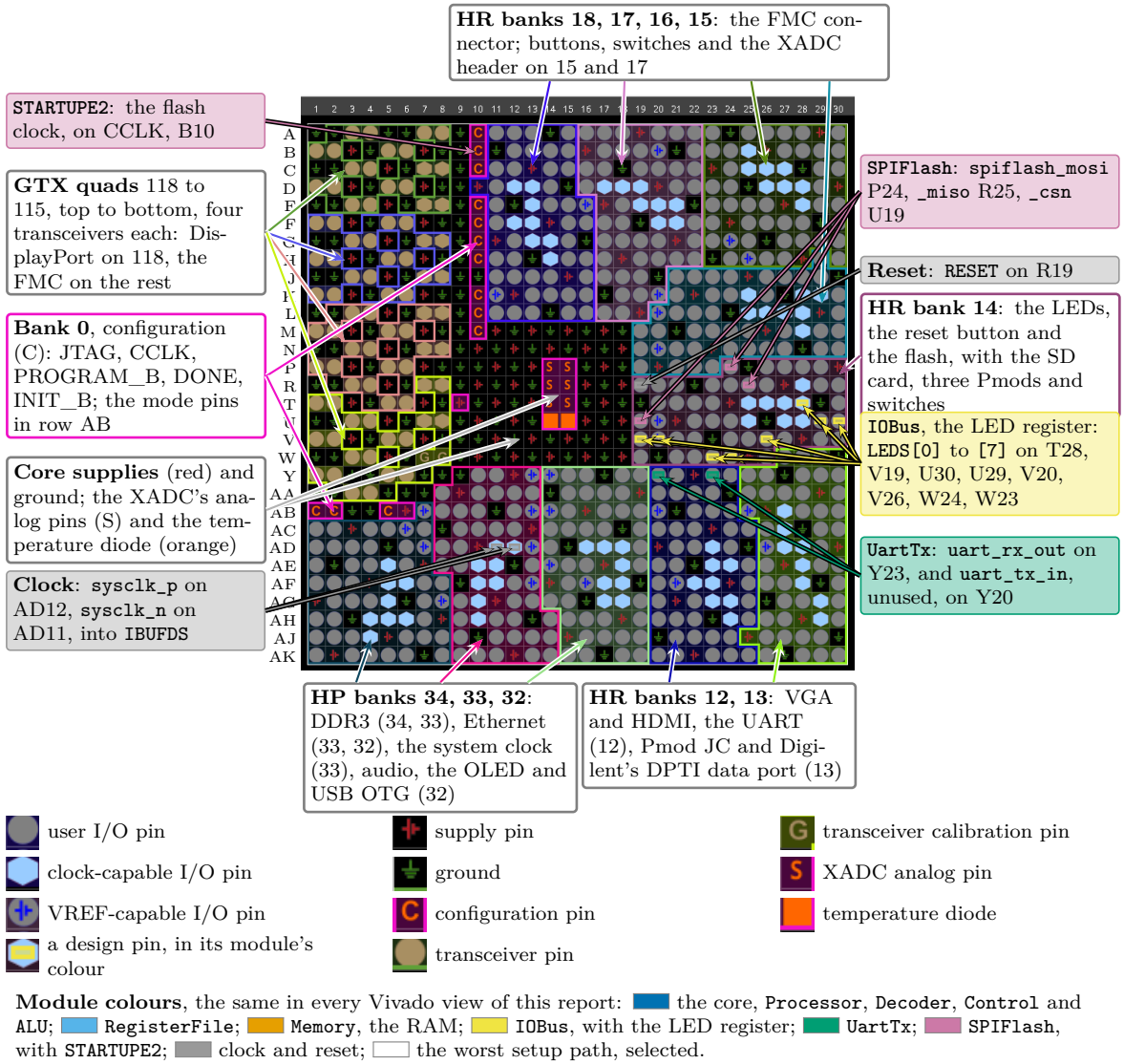


Figure 2.2: Vivado's Package view of the FFG900 package, from the routed design: the 900 balls seen from above, columns 1 to 30 across and rows A to AK down, each drawn as a symbol of the key. Each coloured outline is one bank, and each label says what its banks serve on the Genesys 2 [4]. The design's 16 ports show in their modules' colours, and each module's box points at its pins. The flash's clock has no pin of its own: it leaves on CCLK, a configuration pin. `uart_tx_in`, on Y20, has no load and so no IOB, but keeps its pin.

2.1 Tools

Table 2.2 lists the tools. Vivado's numbers depend on its version: another release may place and route the same RTL differently, so the numbers of Chapter 11 hold for 2025.2.

Tool	Version	Used for
AMD Vivado	2025.2	synthesis, implementation, the xsim regression, the hardware manager (JTAG and flash), the schematics and device views
`riscv64-unknown-elf-gcc`	13.2.0	the firmware, `-march=rv32i -mabi=ilp32` (Ubuntu's `gcc-riscv64-unknown-elf`, with binutils 2.42)
Icarus Verilog	12.0	the ALU unit bench and the waveforms
Python	3.12, Pillow 10.2, pyserial 3.5	figure scripts, board captures
Ghostscript, Poppler	10.02.1, 24.02	cropping figures
pdfTEX	TeX Live 2023	this report

Table 2.2: Tool versions, on Ubuntu 24.04.

3 Repository tour

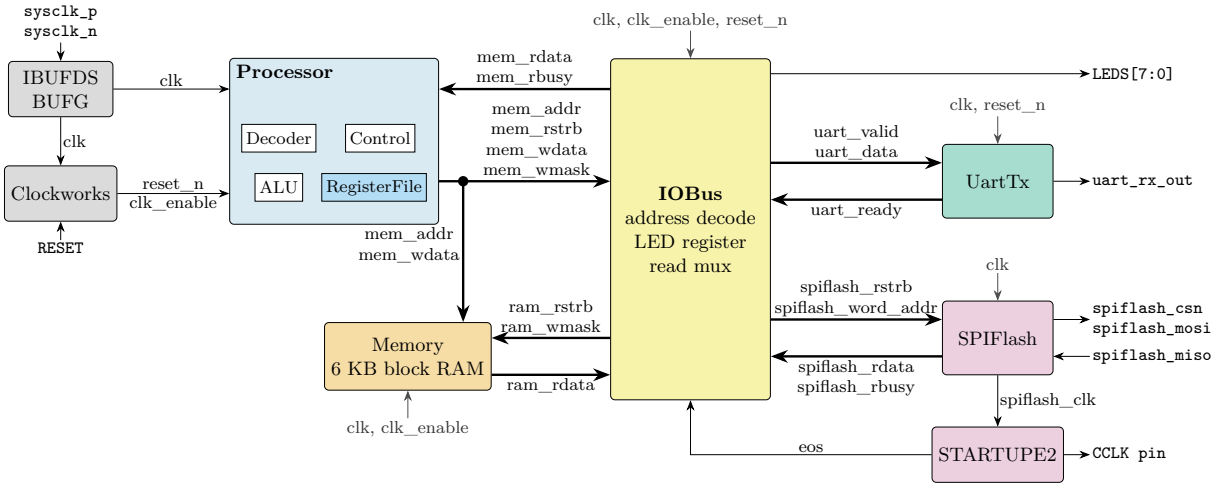
The repository is a plain source tree. The Vivado project is not committed: a script generates it into `build/vivado/`, referring to the sources where they are, and git ignores what the tools generate. Table 3.1 gives each folder at a glance. Each tracked folder has a README of its own, the place to go next: it lists the folder's files, says where to start reading and which make targets serve the folder.

Folder	What it holds
`rtl/`	The synthesisable Verilog: the SoC's top level, the core and the peripherals, one module per file, with the memory map and the firmware image
`fw/`	The demo programs in C and assembly, and what builds them: start-up code, linker scripts, a small runtime and the image converters
`sim/`	The simulation: the SoC testbench, its golden trace and the regression script, and the ALU unit bench
`constraints/`	The board's pins and the 200 MHz clock, for Vivado
`vivado/`	The Tcl scripts that generate the Vivado project and build the bitstream
`docs/`	The coding conventions, the reference numbers, and this report with its figures, scripts, board captures and design notes
`scripts/`	The lint and the git hooks that hold every commit to its gates
`build/`	Everything generated: the Vivado project, `SOC.bit`, the gates' logs. Not tracked

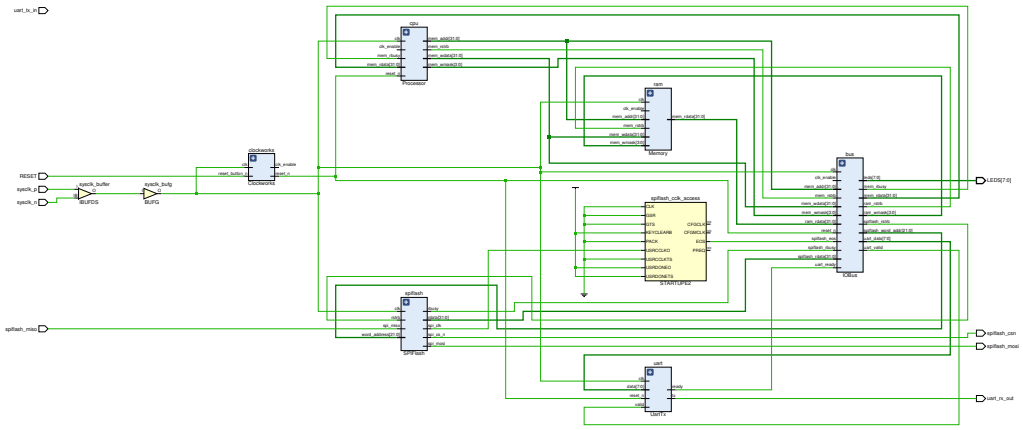
Table 3.1: The folders at a glance; each tracked folder's README says more.

4 System architecture

The SoC is six modules in one clock domain, wired together by `SOC`, the top level, with four more inside the core. The core sees one bus, and everything else hangs off that bus. Figure 4.1 is the block diagram, drawn and as Vivado inferred it, and the sections below give the bus, the memory map and the clock and reset.



(a) The block diagram, from `rtl/SOC.v`, each wire under its name there. The thick arrows are the core's bus and the handshakes `IOBus` derives from it, one arrow for each direction. The core's address and write data are shared wires: at the dot they branch to the RAM as well as to `IOBus`. `IOBus` decodes the address and strobes only the block it selects: the RAM's read strobe and byte mask, the UART's send pulse with the low byte of the write data, or the flash controller's read strobe with the word address. It holds the LED register and multiplexes the read data back to the core. The grey stubs are the left column's `clk`, `clk_enable` and `reset_n`, drawn where they enter. `uart_tx_in`, the line from the PC, is a port of `SOC` that nothing reads, and `STARTUPE2`'s other inputs are constants.



(b) Vivado's RTL schematic of the same top level, elaborated with `synth_design -rtl`: the six instances under their names in `SOC.v`, and the three primitives, `IBUFDS` and `BUFG` on the clock and `STARTUPE2` on the flash clock.

Figure 4.1: The SoC as drawn and as Vivado inferred it.

4.1 The bus and the modules

The core, `Processor`, sees one bus and nothing else: a 32-bit byte address, a one-cycle read strobe, 32 bits of read data, 32 bits of write data with a four-bit byte mask, and a busy input. "One bus" means one set of ports whose far side the core does not know, not that every wire passes through `IOBus`. The address and the write data are plain wires that reach every block that needs them, `IOBus` and the RAM alike, as the dot in Figure 4.1a shows. A block acts on them only when its strobe or its mask says so, and `IOBus` is what raises those for the region the address falls in, and what picks the block whose read data returns to the core. Passing the address and write data through `IOBus` unchanged would add no logic, only a detour in the drawing. There is no acknowledge. A read requested with `mem_rstrb` is answered on `mem_rdata` in the next cycle unless `mem_rbusy` is high, and a write lands on the edge where `mem_wmask` is non-zero. Every address falls in some region, so no access is refused (`notes.md`, "Processor bus has no notion of RAM versus device").

Everything around the core follows from that contract. `IOBus` decodes the address into three regions, forwards the strobe and the mask to the RAM, holds the LED register, turns a store into the UART's send pulse and a flash read into an SPI transaction, and multiplexes the read data back. `Memory` is 6 KB of block RAM with one port, shared by instruction fetch and by loads and stores. Its read is registered, which is why the core waits a cycle after every read, and its write lands on the same edge, which is why a store needs no wait (`notes.md`, "Read and write timing are not symmetric"). `UartTx` sends 8N1 at 115 200 baud and `SPIFlash` reads one 32-bit word per transaction, at a sixth of the system clock. The flash controller is the only block that stalls the core, by holding `mem_rbusy` for the length of a transaction (`notes.md`, "Only flash reads stall the core, read mux"). `Clockworks` is among the instances of Figure 4.1b but contributes no cell to the netlist, as Section 4.3 explains.

4.2 Memory map and address decode

Table 4.1 is the memory map, defined once in `rtl/defines.vh`. The firmware carries copies of its values in `fw/io.h` and the start-up code, and a lint rule holds every copy equal to the defines (Chapter 10).

Byte address	Selected by	Region or register	Access
`0x0000_0000`–`0x0000_17FF`	no region bit	RAM, 6 KB (1536 words)	read, write, fetch
`0x0040_0000`	bit 22	`IO_BASE`, the IO registers	
`+ 0x04`	bit 2	`IO_LEDS`, the LED register	write
`+ 0x08`	bit 3	`IO_UART_DAT`, send one byte	write
`+ 0x10`	bit 4	`IO_UART_CNTL`, status, bit 9 = busy	read
`0x0100_0000`–`0x01FF_FFFF`	bit 24	SPI flash, a 16 MB window on the chip	read, fetch
`0x01D0_0000`–`0x01FF_FFFF`		the code window, 3 MB, where flash boot starts	

Table 4.1: The memory map (`rtl/defines.vh`). The reset vector is `0x0000_0000` in BRAM boot and `0x01D0_0000` in flash boot (Chapter 8).

A region is chosen by one address bit, not by a comparator: bit 24 selects the flash, bit 22 the IO registers when bit 24 is clear, and an address with neither set is RAM. Inside the IO region each register owns one bit as well, so a store to `IO_BASE + 4` lights the LEDs and a store to `IO_BASE + 8` sends a byte. The whole decode is a few bit tests in `IOBus` (`notes.md`, "Address decode: one bit per region, one-hot IO registers, why bit 24"). Bit 24 was chosen with care: everything below the selector bit is the offset inside the flash, and bit 23 would leave 8 MB,

short of the 10.9 MB reserved for a bitstream at the start of the chip, while bit 24 leaves 16 MB, exactly what the 3-byte address of the flash's Read command, opcode `03h`, reaches.

Three consequences matter when writing firmware. Nothing is rejected: an address past the RAM's 6 KB still selects the RAM, and what such an access does is undefined. Only the UART status register answers a read, and every other IO address reads as zero. And a write to flash space does nothing. To add a peripheral, give it the next free bit of the IO word address, bit 5 at byte offset `0x20`, decode it in `IOBus` as the three existing ones are decoded, add it to the read mux if it answers reads, and add its copy in `fw/io.h` to the lint rule.

4.3 Clock and reset

The board's 200 MHz oscillator drives a differential pair, so the clock enters through `IBUFDS` and `BUFG` (`notes.md`, "Genesys 2 differential clock input"). That net, `clk`, is the only clock in the design, and its 5 ns period is the one timing constraint (Chapter 11).

`Clockworks` derives two signals from the clock and the button. `clk_enable` is a gearbox: with the parameter `SLOW` set to n, it is high for one cycle in 2^n, and the core, the bus and the RAM advance only on those cycles, while the UART and the flash controller run every cycle, so the baud rate and the SPI clock stay right (`notes.md`, "Gearbox counter"). The shipped design sets it to 0, `clk_enable` is the constant 1, and the module synthesises to nothing.

The reset input is asynchronous and the resets it drives are synchronous: the two words describe different things. `reset_n` is the CPU RESET button, active-low, forwarded as it is: no synchroniser brings it into the clock domain and no debounce filters it, so it can change at any moment between two clock edges. Every register that resets, though, tests `reset_n` only at a rising edge of `clk`, inside `always @(posedge clk)`, and none has it in its sensitivity list, which is what makes a reset synchronous. Three places have one: the core's `pc`, `instr` and FSM state, the LED register, and the UART's transmitter (`notes.md`, "Synchronous reset and the clock enable"). A press or a release that lands close to an edge can reach one of these registers a cycle before another, or leave one metastable for a moment. A two-flip-flop synchroniser in `Clockworks` would remove that risk, and the design has none. The core's registers also declare their reset value as their initial value, which is what configuration loads into a 7-series flip-flop, so the core leaves configuration in its reset state and starts without a press of the button (`notes.md`, "Power-on state matches the reset state"). In flash boot a press is still needed on the board, for a reason in the flash controller's start-up (pitfall P5). The FSM resets into `WAIT_DATA`, which asserts no strobe and falls through to `FETCH_INSTR`, not into `FETCH_INSTR` itself, and pitfall P6 gives the reason.

5 The RV32I instruction set

RV32I is the base integer instruction set of RISC-V [2], the one this core executes and the one the firmware is compiled for, with `-march=rv32i -mabi=ilp32`. It has 32 registers of 32 bits and 40 instructions, each one 32 bits long. It is a load-store architecture: only the loads and the stores touch memory, and every other instruction works on registers and constants. This chapter gives what the later chapters rely on: the registers, how an instruction is written, how its 32 bits are laid out, what each instruction does in this core, and how a function call uses them.

5.1 Registers and assembly syntax

The registers are `x0` to `x31`. `x0` always reads as zero, and a write to it is discarded, which makes it useful as a source of zero and as a destination for a result nobody needs. The program counter, `pc`, is separate and is not one of the 32. The calling convention (Section 5.4) gives each register a role and an ABI name, the name the compiler and the disassembler print (Table 5.1).

Registers	ABI names	Role
`x0`	`zero`	always zero
`x1`	`ra`	return address, written by a call
`x2`	`sp`	stack pointer, the top of the RAM at start-up
`x3`	`gp`	global pointer, set to `IO_BASE` by this design's start-up code
`x4`	`tp`	thread pointer, unused here
`x5`–`x7`, `x28`–`x31`	`t0`–`t2`, `t3`–`t6`	temporaries, which a call may overwrite
`x8`, `x9`, `x18`–`x27`	`s0`–`s1`, `s2`–`s11`	saved registers, which a call preserves (`s0` also `fp`)
`x10`–`x17`	`a0`–`a7`	arguments, `a0` and `a1` also the return value

Table 5.1: The 32 registers and their roles in the calling convention.

5.2 Instruction formats

Every instruction is one of six formats, R, I, S, B, U and J, which differ in where they keep their constant, the *immediate* (Figure 5.1). The fields are:

- **opcode**, bits 6 to 0, the instruction's class: register arithmetic, arithmetic with an immediate, load, store, branch and so on. Its two low bits are `11` in every 32-bit instruction.
- **rd**, **rs1** and **rs2**, five bits each, the numbers of the destination and the two source registers. They sit at the same bit positions in every format that has them, so the register file can be read before the instruction is fully decoded. This core reads `rs1` and `rs2` in `FETCH_REGS` for every instruction, whether it uses them or not (Chapter 6).
- **funct3** and **funct7**, which choose the operation within a class: `funct3` picks one of the eight ALU operations, the branch condition or the load and store width, and bit 30, in `funct7`, tells `SUB` from `ADD` and `SRA` from `SRL`.
- **imm**, the immediate, a signed constant. Its sign is always bit 31, so it can be sign-extended before the format is known. I and S immediates are 12 bits, from -2048 to 2047. B and J immediates are byte offsets with bit 0 left out, since every instruction starts at an even address: a branch reaches ± 4 KB and `jal` ± 1 MB. A U immediate is the upper 20 bits of a 32-bit value.

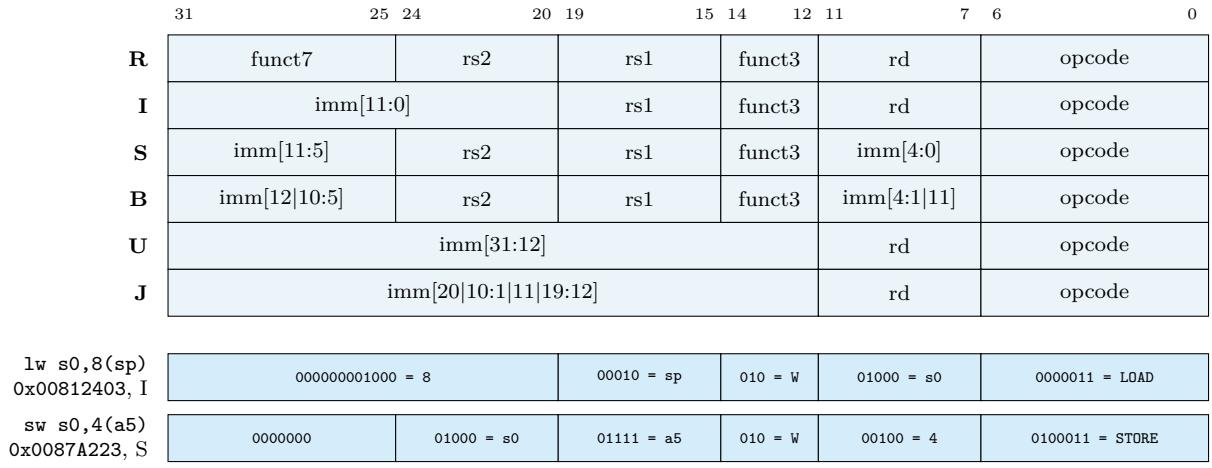


Figure 5.1: The six instruction formats, bit 31 on the left, and two instructions of the mandelbrot demo decoded: the load of Figure 6.3 and the store that writes the LEDs in Figure 9.1. `imm[12|10:5]` means immediate bit 12, then bits 10 to 5. B and J scatter their immediate's bits so that each immediate bit sits at the same instruction bit in as many formats as possible.

5.3 The instructions

Table 5.2 lists the 40 instructions, one a row, with what each one does and how many cycles it takes in this core when the instruction is fetched from the RAM (Table 6.1). In the operations, `imm` is the sign-extended immediate, `mem` the memory, and `x0` the register that reads as zero. RV32I has no multiply and no divide: the compiler turns every `*`, `/` and `%` in C into a call into `libgcc`, which does it with shifts and adds. After the instructions, the table lists the pseudo-instructions and aliases that the GNU assembler (binutils 2.42, `-march=rv32i`) accepts, each with the real instructions it writes in its place, as `objdump -M no-aliases` shows them, and so the cycles they cost. The paragraph after the table says why the firmware can hold nothing else.

Table 5.2: The RV32I instructions and what this core does with each, then the pseudo-instructions and aliases of the GNU assembler. Fmt is the format of Figure 5.1, and the cycles are from the RAM: every word fetched or loaded from the flash adds an SPI transaction (Chapter 7).

Instruction	Fmt	Operation	Cycles
Register and register			
`ADD rd,rs1,rs2`	R	`rd` ← `rs1` + `rs2`	4
`SUB rd,rs1,rs2`	R	`rd` ← `rs1` − `rs2`	4
`AND rd,rs1,rs2`	R	`rd` ← `rs1` AND `rs2`, bit by bit	4
`OR rd,rs1,rs2`	R	`rd` ← `rs1` OR `rs2`, bit by bit	4
`XOR rd,rs1,rs2`	R	`rd` ← `rs1` XOR `rs2`, bit by bit	4
`SLL rd,rs1,rs2`	R	`rs1` shifted left by `rs2[4:0]` places, zeros in	4
`SRL rd,rs1,rs2`	R	`rs1` shifted right by `rs2[4:0]` places, zeros in	4
`SRA rd,rs1,rs2`	R	`rs1` shifted right by `rs2[4:0]` places, copies of its sign bit in	4
`SLT rd,rs1,rs2`	R	`rd` ← 1 if `rs1` < `rs2` as signed numbers, else 0	4
`SLTU rd,rs1,rs2`	R	`rd` ← 1 if `rs1` < `rs2` as unsigned numbers, else 0	4
Register and immediate			
`ADDI rd,rs1,imm`	I	`rd` ← `rs1` + `imm`	4
`ANDI rd,rs1,imm`	I	`rd` ← `rs1` AND `imm`, bit by bit	4
`ORI rd,rs1,imm`	I	`rd` ← `rs1` OR `imm`, bit by bit	4
`XORI rd,rs1,imm`	I	`rd` ← `rs1` XOR `imm`, bit by bit	4
`SLLI rd,rs1,sh`	I	`rs1` shifted left by `sh`, 0 to 31 places, zeros in	4
`SRLI rd,rs1,sh`	I	`rs1` shifted right by `sh`, 0 to 31 places, zeros in	4

Instruction	Fmt	Operation	Cycles
`SRAI rd,rs1,sh`	I	`rs1` shifted right by `sh`, 0 to 31 places, copies of its sign bit in	4
`SLTI rd,rs1,imm`	I	`rd` ← 1 if `rs1` < `imm` as signed numbers, else 0	4
`SLTIU rd,rs1,imm`	I	`rd` ← 1 if `rs1` < `imm` as unsigned numbers, else 0	4
Upper immediate			
`LUI rd,imm`	U	`rd` ← `imm` ≪ 12	4
`AUIPC rd,imm`	U	`rd` ← `pc` + (`imm` ≪ 12)	4
Jumps			
`JAL rd,label`	J	`rd` ← `pc` + 4, `pc` ← `pc` + `imm`: a call when `rd` is `ra`	4
`JALR rd,imm(rs1)`	I	`rd` ← `pc` + 4, `pc` ← `rs1` + `imm` with bit 0 cleared: a return, or a call through a pointer	4
Branches			
`BEQ rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` = `rs2`	4
`BNE rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` ≠ `rs2`	4
`BLT rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` < `rs2` as signed numbers	4
`BGE rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` ≥ `rs2` as signed numbers	4
`BLTU rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` < `rs2` as unsigned numbers	4
`BGEU rs1,rs2,label`	B	`pc` ← `pc` + `imm` if `rs1` ≥ `rs2` as unsigned numbers	4
Loads			
`LW rd,imm(rs1)`	I	`rd` ← the word at `rs1` + `imm`	6
`LH rd,imm(rs1)`	I	`rd` ← the halfword at `rs1` + `imm`, sign-extended	6
`LHU rd,imm(rs1)`	I	`rd` ← the halfword at `rs1` + `imm`, zero-extended	6
`LB rd,imm(rs1)`	I	`rd` ← the byte at `rs1` + `imm`, sign-extended	6
`LBU rd,imm(rs1)`	I	`rd` ← the byte at `rs1` + `imm`, zero-extended	6
Stores			
`SW rs2,imm(rs1)`	S	the word at `rs1` + `imm` ← `rs2`	5
`SH rs2,imm(rs1)`	S	the halfword at `rs1` + `imm` ← the low halfword of `rs2`, the rest of the word left as it was	5
`SB rs2,imm(rs1)`	S	the byte at `rs1` + `imm` ← the low byte of `rs2`, the rest of the word left as it was	5
Ordering and system			
`FENCE`	I	orders memory accesses between cores and devices. Here a no-op	4
`ECALL`	I	calls the operating system. Here it stops the core	4, repeated
`EBREAK`	I	calls the debugger. Here it stops the core	4, repeated
Pseudo-instructions and aliases: what the assembler writes for each			
`nop`	–	`ADDI x0,x0,0`	4
`li rd,imm`	–	`ADDI rd,x0,imm` when `imm` fits in 12 bits, −2048 to 2047. Otherwise `LUI rd,hi` then `ADDI rd,rd,lo`: `lo` is the low 12 bits of `imm` read as a signed number, `hi` is (`imm` − `lo`) ≫ 12, and the `ADDI` is left out when `lo` is 0. For example, `li a0,0x12345fff` is `LUI a0,0x12346` then `ADDI a0,a0,-1`	4 or 8
`la rd,symbol`	–	`AUIPC rd` then `ADDI rd,rd`: the symbol's address, without position-independent code, as the firmware is built	8
`lla rd,symbol`	–	`AUIPC rd` then `ADDI rd,rd`	8
`mv rd,rs`	–	`ADDI rd,rs,0`	4
`not rd,rs`	–	`XORI rd,rs,-1`	4
`neg rd,rs`	–	`SUB rd,x0,rs`	4
`seqz rd,rs`	–	`SLTIU rd,rs,1`: 1 if `rs` is zero	4
`snez rd,rs`	–	`SLTU rd,x0,rs`: 1 if `rs` is not zero	4
`sltz rd,rs`	–	`SLT rd,rs,x0`: 1 if `rs` is negative	4
`sgtz rd,rs`	–	`SLT rd,x0,rs`: 1 if `rs` is positive	4
`zext.b rd,rs`	–	`ANDI rd,rs,255`	4
`sext.b rd,rs`	–	`SLLI rd,rs,24` then `SRAI rd,rd,24`	8
`sext.h rd,rs`	–	`SLLI rd,rs,16` then `SRAI rd,rd,16`	8

Instruction	Fmt	Operation	Cycles
`zext.h rd,rs`	–	`SLLI rd,rs,16` then `SRLI rd,rd,16`	8
`beqz rs,label`	–	`BEQ rs,x0,label`	4
`bnez rs,label`	–	`BNE rs,x0,label`	4
`blez rs,label`	–	`BGE x0,rs,label`	4
`bgez rs,label`	–	`BGE rs,x0,label`	4
`bltz rs,label`	–	`BLT rs,x0,label`	4
`bgtz rs,label`	–	`BLT x0,rs,label`	4
`bgt rs,rt,label`	–	`BLT rt,rs,label`	4
`ble rs,rt,label`	–	`BGE rt,rs,label`	4
`bgtu rs,rt,label`	–	`BLTU rt,rs,label`	4
`bleu rs,rt,label`	–	`BGEU rt,rs,label`	4
`j label`	–	`JAL x0,label`	4
`jal label`	–	`JAL ra,label`	4
`jr rs`	–	`JALR x0,0(rs)`	4
`jalr rs`	–	`JALR ra,0(rs)`	4
`ret`	–	`JALR x0,0(ra)`	4
`call label`	–	`AUIPC ra` then `JALR ra,0(ra)`	8
`tail label`	–	`AUIPC t1` then `JALR x0,0(t1)`: a jump that does not return	8
`fence`	–	`FENCE iorw,iorw`: every kind of access ordered	4
`lw rd,symbol`	–	`AUIPC rd` then `LW rd,0(rd)`	10
`lh rd,symbol`	–	`AUIPC rd` then `LH rd,0(rd)`	10
`lhu rd,symbol`	–	`AUIPC rd` then `LHU rd,0(rd)`	10
`lb rd,symbol`	–	`AUIPC rd` then `LB rd,0(rd)`	10
`lbu rd,symbol`	–	`AUIPC rd` then `LBU rd,0(rd)`	10
`sw rs,symbol,rt`	–	`AUIPC rt` then `SW rs,0(rt)`	9
`sh rs,symbol,rt`	–	`AUIPC rt` then `SH rs,0(rt)`	9
`sb rs,symbol,rt`	–	`AUIPC rt` then `SB rs,0(rt)`	9
`unimp`	–	`CSRRW x0,cycle,x0`, a SYSTEM instruction of Zicsr: here it stops the core	4, repeated

Pseudo-instructions. A pseudo-instruction exists only in assembly text. The assembler replaces it with the real instructions of its row before any binary exists, so the core never sees one. The firmware is assembled by the GNU assembler, not by a tool of this project, and every row of the table was assembled with `-march=rv32i` and disassembled to check what it becomes. What keeps other instructions out of the binary is the `-march=rv32i -mabi=ilp32` that `fw/Makefile` passes to every compile and every assembly. With it the compiler emits only RV32I instructions and these pseudo-instructions, and the assembler refuses any instruction of an extension that `-march` does not name: `mul a0,a1,a2` stops the build with "extension 'm' or 'zmmul' required". `libgcc` and `libm` are linked from their builds for `rv32i/ilp32`. A disassembly of every demo's ELF shows RV32I instructions only.

FENCE. Its opcode sets none of `Decoder`'s class flags, so the core treats it as an ALU instruction whose `rd` is `x0`: a no-op, which is all a `FENCE` has to do with one core, no cache and memory accessed in program order. `FENCE.I` shares the opcode and is a no-op too, also correctly: instructions and data come from the same memory, uncached.

ECALL and EBREAK. `EXECUTE` leaves `pc` unchanged for the SYSTEM opcode, so the core fetches the instruction again, forever: a halt that the reset button ends. The start-up code ends with `EBREAK`, should `main` return.

Left out. No extension: no M, A, F, D or C, and no Zicsr. The CSR instructions share the SYSTEM opcode and stop the core as `ECALL` does. There are no privilege modes, no exceptions

and no interrupts: an illegal instruction is executed as `FENCE` is, and a misaligned load or store is not trapped, so its result is wrong, though code built from C never makes one for aligned data.

5.4 The calling convention

RV32I has no call, return, push or pop instruction: a call is a `JAL` or `JALR` that writes the return address to `ra`, a return is a `JALR` through `ra`, and the stack is `sp` and the loads and stores. Everything else about a call is the calling convention of the RISC-V ABI for `ilp32`, which the compiler follows and the hand-written assembly in `fw/` follows too, so that C and assembly call each other. It divides the work between the two sides of a call:

- **The caller** puts the first eight arguments in `a0` to `a7`, any more on the stack, and calls with `call`, `AUIPC` then `JALR`, which leaves the return address in `ra`. After the call the result is in `a0`, and `a1` too for a 64-bit one. The caller must assume that the call has overwritten `ra`, `t0` to `t6` and `a0` to `a7`, the *caller-saved* registers, so a value it needs after the call it keeps in an `s` register or in its own stack frame.
- **The callee** may overwrite the caller-saved registers, but must return with `sp` and `s0` to `s11`, the *callee-saved* registers, as it found them. A function that uses an `s` register saves it first, and so does one that calls another function, which overwrites `ra`. Its *prologue* takes a stack frame, by subtracting from `sp`, and stores those registers in it; its *epilogue* loads them back, gives the frame back and returns with `ret`.

The stack grows down from the top of the RAM, where the start-up code points `sp`, and `sp` stays a multiple of 16 bytes, so every frame is too. Table 5.3 is one function of the mandelbrot demo as the compiler made it, `print_string` of `fw/print.c`, which hands each character of a string to `putchar`. Its pointer cannot stay in `a0`, which each call to `putchar` may overwrite, so it lives in `s0`, which `putchar` must preserve; `print_string` owes its own caller the same, so its prologue saves `s0`, and `ra`, which its calls overwrite. A *leaf* function, one that calls nothing and needs no `s` register, needs neither prologue nor epilogue: `putchar`, in `fw/putchar.S`, loops in `t0` and `t1` and ends in `ret`. At the other end, `main` saves `ra` and all twelve `s` registers in a 96-byte frame that also holds its locals, among them the frame counter that Figure 6.3 loads from `8(sp)`. In this core a call and its return take 12 cycles, `AUIPC`, `JALR` and `ret`, and the prologue and epilogue of Table 5.3 30 more (Table 5.2). `gp` plays no part: the convention keeps it for the linker, and the firmware is linked with `--no-relax`, so the linker leaves it alone and the start-up code sets it to `IO_BASE` for the IO registers.

Address	Instruction	What it does
Prologue		
42c	`addi sp,sp,-16`	takes a 16-byte frame
430	`sw s0,8(sp)`	saves the caller's `s0`, which the body uses
434	`sw ra,12(sp)`	saves the return address, which the calls overwrite
Body		
438	`mv s0,a0`	keeps the pointer, the first argument, where `putchar` preserves it
43c	`lbu a0,0(a0)`	the first character, as `putchar`'s argument
440	`beqz a0,458`	an empty string: straight to the epilogue
444	`auipc ra,0x0`	`call putchar`, first half: `ra` = `pc` = `0x444`
448	`jalr -68(ra)`	second half: to `0x444` − 68 = `0x400`, `putchar`, with `ra` = `0x44c`
44c	`lbu a0,1(s0)`	the next character
450	`addi s0,s0,1`	
454	`bnez a0,444`	and again, until the terminating zero
Epilogue		
458	`lw ra,12(sp)`	restores the return address
45c	`lw s0,8(sp)`	restores the caller's `s0`
460	`addi sp,sp,16`	gives the frame back
464	`ret`	`JALR x0,0(ra)`: back to the caller

Table 5.3: `print_string` of `fw/print.c` in the mandelbrot demo, from `objdump -d mandelbrot.bram.elf`: `for(const char* p = s; *p; ++p) putchar(*p);`, compiled with `-O2`.

6 The multi-cycle core

`Processor` executes RV32I and nothing beyond it: no M extension, no interrupts, no control and status registers, and `ECALL` and `EBREAK` stop the core (Chapter 5). It is a multi-cycle core: one instruction at a time, several clock cycles each, one FSM state per cycle. `Control` is the FSM, `Decoder`, `ALU` and `RegisterFile` are what their names say, and `Processor` holds `pc`, the instruction register and the glue between the other four.

6.1 The state machine

An instruction takes several cycles because the RAM answers a read one cycle after it is requested, and the register file's read is synchronous too, so each needs a state of its own (`notes.md`, "Why one instruction takes several cycles"). Figure 6.1 shows the seven states.

`FETCH_INSTR` drives `pc` on the bus with the read strobe, `WAIT_INSTR` latches the word the RAM returns, `FETCH_REGS` latches `rs1` and `rs2`, and `EXECUTE` loads `pc` with its next value and writes `rd` for every instruction but a load, a store or a branch. A load goes on to `LOAD`, which drives the data address with the strobe, and `WAIT_DATA`, which writes the returned word to `rd`. A store goes to `STORE`, which drives the address, the data and the byte mask for one cycle. The RAM writes on that same edge, so there is no wait after a store (`notes.md`, "STORE needs no wait state"). The two waits are the only states that can last longer than a cycle: they hold while `mem_rbusy` is high, which only the flash controller raises (`notes.md`, "Instruction fetch stalls on mem_rbusy").

`Control` decodes four one-cycle strobes from the state, `latch_instr`, `read_regs`, `update_pc` and `write_rd`, already qualified with `clk_enable` and `reset_n`. `write_rd` in `WAIT_DATA` is gated by `mem_rbusy` as well, or a stalled flash load would write the bus's garbage to `rd` on every stalled cycle (`notes.md`, "Load stalls on mem_rbusy"). Table 6.1 gives the cost per instruction class.

Instruction class	States	Cycles
ALU with a register or an immediate, LUI, AUIPC, JAL, JALR, branch	FI WI FR EX	4
Load: LB, LH, LW, LBU, LHU	FI WI FR EX LD WD	6
Store: SB, SH, SW	FI WI FR EX ST	5
SYSTEM: ECALL, EBREAK	FI WI FR EX, `pc` unchanged	4, repeated

Table 6.1: Cycles per instruction class from the RAM. A SYSTEM instruction leaves `pc` alone, so the core fetches it again, forever. Every word fetched or loaded from the flash adds the SPI transaction to its wait state (Chapter 7).

Two of the four cycles of the commonest class are the fetch. A pipelined core would overlap the fetch of one instruction with the execution of the last and retire close to one a cycle, a gain of four to six on code bound by computation. The demos of Section 1.1 are bound by the UART, so for them the multi-cycle core costs nothing visible. Pipelining is the first extension in Chapter 12.

6.2 The datapath

`Decoder` is purely combinational on the instruction register: the ten class flags from the opcode, disjoint, so exactly one is high for every instruction the core implements, the register indices and the `funct3` and `funct7` fields, and the five immediates of Figure 5.1, sign-extended from

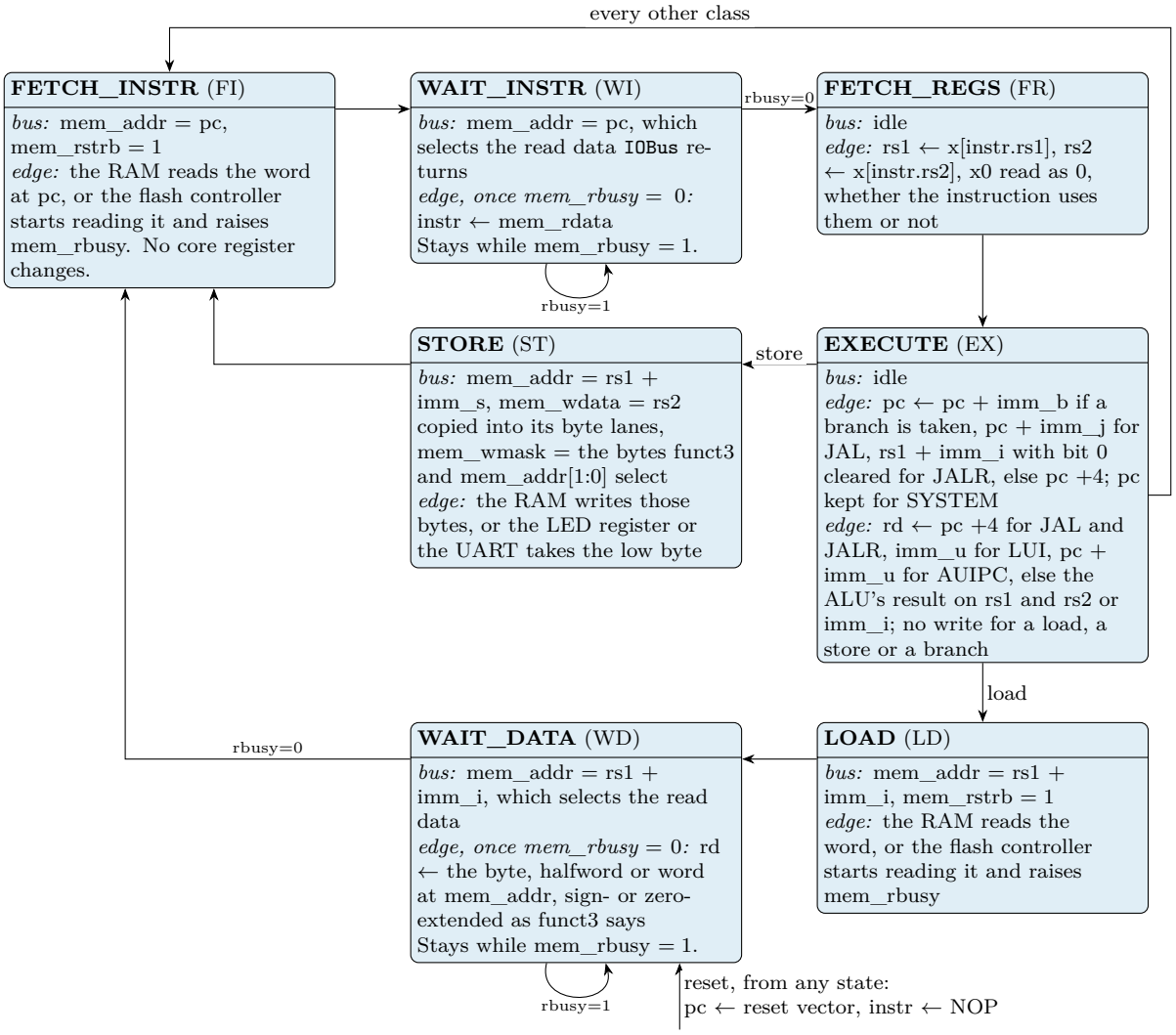


Figure 6.1: The control FSM of `rtl/Control.v` and what each state does to the datapath of `rtl/Processor.v`: *bus* is what the core drives on the bus in that state, *edge* what the clock edge that ends the state writes. Every state lasts one cycle but the two waits, which hold while the flash controller is busy. The two-letter codes are the ones Figure 6.3 uses. Reset puts the FSM in `WAIT_DATA`, which asserts no strobe, so a core held in reset leaves the bus idle. Its write to rd goes to `x0`, the destination of the NOP that instr resets to, and when reset ends it falls through to `FETCH_INSTR` at the reset vector (pitfall P6).

bit 31. It also settles the two meanings of `funct7[5]`: a subtraction only in the register form, since an `ADDI` whose immediate happens to set bit 30 must still add, and an arithmetic shift in both forms (`notes.md`, "ADD versus SUB: funct7[5] alone is not enough").

`RegisterFile` is the 32 integer registers in LUT RAM: a synchronous read of `rs1` and `rs2` on `read_regs`, a synchronous write of `rd` on `write_rd`, and `x0` never written and read as zero explicitly (`notes.md`, "x0 reads as zero explicitly"). `ALU` is combinational (Figure 6.2): its second operand is `rs2` for register instructions and the I-immediate for everything else, `funct3` picks one of the eight operations, and a second case on `funct3` computes the six branch conditions, with a default of zero so that no latch is inferred (`notes.md`, "Branch comparator default arm").

Both arms of the right shift are signed, or `SRA` shifts logically (pitfall P4).

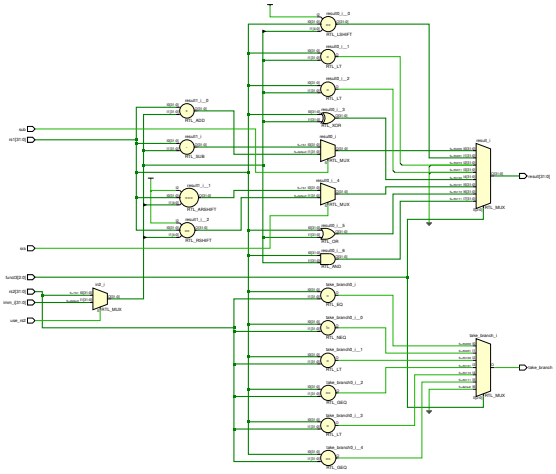


Figure 6.2: Vivado's RTL schematic of `rtl/ALU.v`, elaborated as its own top. Upper half: the operand multiplexer on the left, then the adder and subtractor, the shifters, the comparators and the logic operations, into the result multiplexer on the right. Lower half: the branch comparators, into the multiplexer that yields `take_branch`. After synthesis nearly all of this merges into the LUTs around the register file (Chapter 11).

The rest of `Processor` is the glue between these. A multiplexer chooses the next `pc`: the branch target when a branch is taken, the jump target for `JAL`, `rs1` plus the immediate with bit 0 cleared for `JALR`, otherwise `pc + 4`. An adder forms the load or store address from `rs1` and the I- or S-immediate. The memory moves whole words, so a byte or halfword load picks its lane by the address's low two bits and sign-extends unless `funct3[2]` says not to, and a store broadcasts `rs2` into every lane it could land in and lets the byte mask select (`notes.md`, "Store data is broadcast to every lane, the mask selects"). The write-back multiplexer gives `rd` its value: `pc + 4` for the jumps, the U-immediate for `LUI`, `pc` plus it for `AUIPC`, the aligned load data, or the ALU result. The design's worst timing path runs through this glue, from the instruction register through the load-store address adder, the region decode and the load lanes to the register file's write port (pitfall P7).

6.3 One instruction through the states

Figure 6.3 is the simulated core at the end of `main`'s prologue in the mandelbrot demo, where the frame counter, just stored to the stack, is loaded back and written to the LEDs: four instructions, 19 cycles, every state of Figure 6.1.

The load takes cycles 0 to 5. In `FETCH_INSTR` the bus address is `pc`, `0x278`, with the strobe. In `WAIT_INSTR` the RAM answers with the instruction word, `0x00812403` (Figure 5.1), and `latch_instr` moves it into `instr`. From then on the bus address is the load-store address, `sp + 8`. `EXECUTE` advances `pc` to `0x27c`, `LOAD` asserts the strobe at the data address, and `WAIT_DATA` sees the word, 0, and `write_rd` writes it to `s0`.

`mem_addr` is `pc` in `FETCH_INSTR` and `WAIT_INSTR`, and the load-store address, `rs1` plus the immediate, in every other state, whatever the instruction. It matters only where the core uses the bus, the cycles marked in Figure 6.3: with the read strobe in `FETCH_INSTR` and `LOAD` and in the wait state after each, where `IOBus` picks the returned word by the address's region bits, and with the byte mask in `STORE`. Here those are `0x278`, `0x27c`, `0x280` and `0x284` for the four fetches, `0x17a8`, `sp + 8`, for the load and `0x400004` for the store. In the other cycles the address is the adder's sum of whatever `rs1` and the instruction's immediate bits hold, and nothing uses it: `0x17a4` and `0x4` during the first `LUI` are `sp`, still in `rs1`, and then `x0`, each plus 4, the `LUI`'s

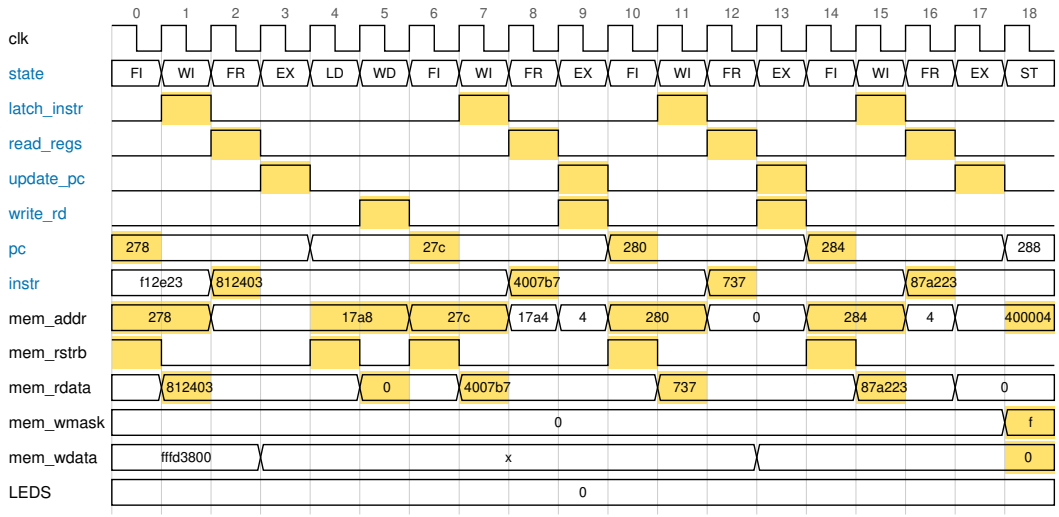


Figure 6.3: Four instructions of the mandelbrot demo in the simulated SoC, from the VCD of `docs/design/scripts/waves/WaveBench.v`: `lw s0,8(sp)` at `0x278`, `lui a5,0x400`, `lui a4,0x0` and `sw s0,4(a5)`, the store to the LED register. The core's signals are in blue, the bus in black. Bus values are in hex with leading zeros dropped. In each state the values to read, those the state uses or produces, are marked in yellow; an unmarked value is one nothing uses in that cycle, as `mem_addr` is in `FETCH_REGS` and `EXECUTE`. `mem_wdata` reads `x` while the instruction's `rs2` field names a register no instruction has written yet: the register file has no reset.

bits taken as an I-immediate. The two `LUI`s take four cycles each, writing `rd` in `EXECUTE`. The store, cycles 14 to 18, writes no register. In `STORE` the bus carries the LED register's address, `0x400004`, the byte mask `f` and the data, and the register takes the value on the edge that ends the window. Each read is answered one cycle after its strobe, and no cycle is spent on the store: the two facts about the memory that shaped the FSM.

7 Memory and memory-mapped IO

To the core the three regions of the bus differ only in timing. The RAM and the IO registers answer every read in the next cycle and take every write on the edge it is presented, while the flash answers a read about 380 cycles later, and the core waits.

7.1 The RAM

`Memory` is 1536 words, 6 KB, of block RAM: two of the Kintex-7's 445 tiles. Its one port serves instruction fetch, loads and stores alike, so in BRAM boot the program, its data and its stack share it, the stack growing down from the top. The read is registered: the word addressed in the cycle of `ram_rstrb` appears on `ram_rdata` at the next edge, which is the one cycle every wait state of the FSM spends. A write needs no strobe and no wait: each of the four `ram_wmask` bits writes its byte lane on the edge where it is high, so `SB` and `SH` leave the rest of the word as it was (`notes.md`, "Per-byte write enables"). `IOBus` masks both strobes off outside RAM space, so a store to an IO register never also writes the RAM word with the same index (`notes.md`, "RAM strobes masked outside RAM space").

The RAM's contents at configuration are the firmware image, `rtl/firmware.hex`, read with `$readmemh` when the bitstream is synthesised, so a BRAM-boot program is part of the bitstream and a new program is a new bitstream. The file is read by bare name, relative to the working directory of the tool, and a missing file leaves the RAM undefined with no error: pitfall P2 (`notes.md`, "Bare file name in $readmemh").

7.2 The LED register and the UART

The IO registers are three addresses in the IO region, and none of them is memory: a store to one is an action and a load from one is a question.

LEDs. A store to `IO_BASE + 4` sets the LED register to the low byte of the store data. `SW` and `SB` both work, since `IOBus` ignores the byte mask. The register drives `LD0` to `LD7` directly, clears on reset, and reads as zero (`notes.md`, "LED register: reset not gated by the clock enable").

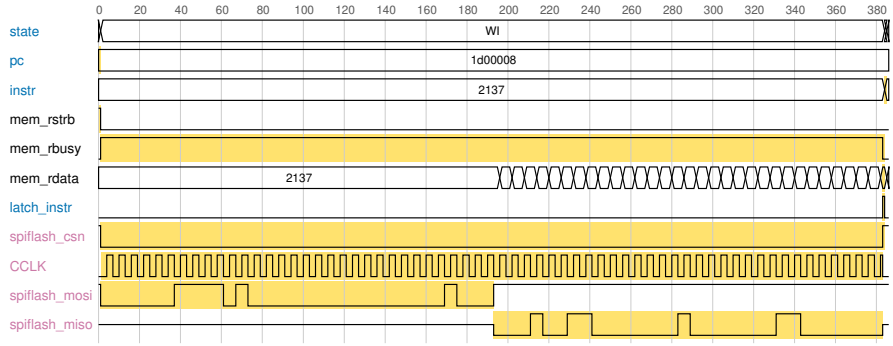
UART data. A store to `IO_BASE + 8` turns into `uart_valid`, a one-cycle pulse, with the low byte as `uart_data`. `UartTx` drops `ready` on the same edge and sends the byte as one start bit, eight data bits LSB first and one stop bit, each 1736 cycles long: 115 207 baud, 0.006 % above 115 200. A byte takes 17 360 cycles, 86.8 µs. The transmitter has no FIFO and ignores `valid` while it sends, so a byte stored too early is lost without a trace (`notes.md`, "UART send pulse and the status word (bit 9 convention)").

UART status. A load from `IO_BASE + 0x10` returns a word that is zero but for bit 9, which is `!ready`: the transmitter is busy. It is the only IO address that answers a read, and bit 9 is a convention between `IOBus` and the firmware, held equal by the memory-map lint rule. `fw/putchar.S` is the whole driver: it stores the byte, then loads the status word until bit 9 is clear, which is why the demos run at the pace of the UART.

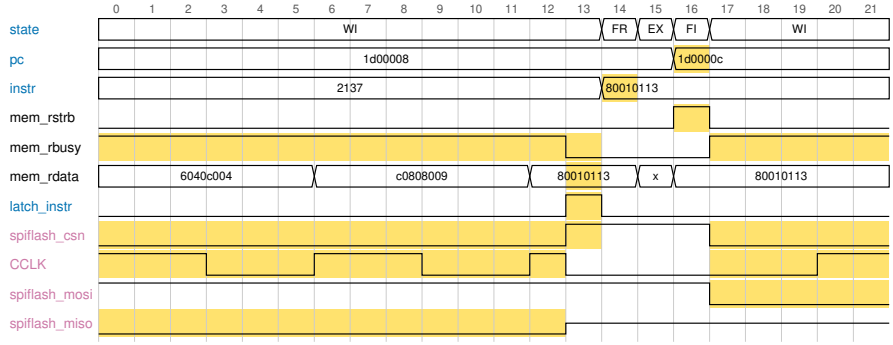
7.3 The flash stall

A read in flash space is the one access the core waits for. In the cycle of the read strobe, `IOBus` passes it to `SPIFlash` with the word address, and the controller drops chip select and raises

`mem_rbusy` at the next edge, and holds both for the whole SPI transaction. The FSM's two wait states hold while `mem_rbusy` is high, and `write_rd` in `WAIT_DATA` is gated by it, so a stalled load writes nothing until its word has arrived (`notes.md`, "Only flash reads stall the core, read mux"). Figure 7.1 is one such stall, simulated: the third instruction of flash-boot start-up, fetched from `0x1D00008`.



(a) The whole instruction, 386 cycles: `FETCH_INSTR` at cycle 0, `WAIT_INSTR` for 383 cycles, then `FETCH_REGS` and `EXECUTE`.



(b) The end of the same transaction, 22 cycles from the last two data bits: chip select rises, `mem_rbusy` falls, the word is latched, and the next fetch starts the next transaction.

Figure 7.1: An instruction fetched from the flash, in the simulated SoC in flash boot (`docs/design/scripts/waves/WaveBench.v` with a model of the flash's read command, booting mandelbrot's flash build). The core's signals are in blue, the flash controller's pins in purple. The values to read in each state are marked in yellow: `pc` and the strobe in `FETCH_INSTR`. In `WAIT_INSTR`, `mem_rbusy` and chip select, which frame the transaction; `CCLK` while it clocks; `spiflash_mosi` while the command and address go out and `spiflash_miso` while the word comes in; and `mem_rdata` with `latch_instr` in the one cycle the word is taken. Then the new `instr` in `FETCH_REGS`. `CCLK` is the flash's clock pin as Vivado's model of `STARTUPE2` drives it. Unmarked, `mem_rdata` shows the controller's shift register, byte-swapped, while the word shifts in, and `spiflash_miso` is drawn mid-way where the flash leaves it undriven.

The transaction is 64 SPI clocks at a sixth of the system clock, 33.3 MHz: the Read command's opcode, `03h`, and the 24-bit byte address on `spiflash_mosi`, then 32 data bits on `spiflash_miso`. That is 382 cycles of `mem_rbusy`, and the instruction takes 386 cycles, 1.93 µs, where one fetched from the RAM takes 4. The word arrives lowest address first, each byte MSB first, as `13 01 01 80`, and the controller swaps the bytes into `0x80010113`, `addi sp,sp,-2048`, little-endian as the RAM returns it (`notes.md`, "Byte order of the received word"). Chip select rises right after the last bit, and `mem_rbusy` with it. In that cycle `WAIT_INSTR` latches the word and the core moves on. The controller's clock and its divider are held while chip select is high, so the first clock pulse of a transaction is as long as the others (`notes.md`, "Divider held in reset while idle (short first pulse)"). There is no cache and no prefetch: every word read from the

flash is a transaction of its own, so code runs from the flash about a hundred times slower than from the RAM, which is what shapes flash boot in the next chapter.

8 Boot modes and the SPI flash

The core starts at its reset vector, the `SOC` parameter `BOOT_ADDR`, and that one parameter is the boot mode. At address 0 the core runs a program held in the RAM, BRAM boot. At `0x01D0_0000` it runs a program stored in the flash, in place, flash boot. The hardware is otherwise the same (`notes.md`, "Why the reset vector is a parameter, and why a jump at address 0 is not enough"). Table 8.1 compares the two.

	BRAM boot	Flash boot
Reset vector	`0x0000_0000`	`0x01D0_0000`, the default
Program held in	the RAM, from `rtl/firmware.hex`	the flash's code window
Room for code	6 KB, shared with data and stack	3 MB; data and stack in the RAM
A new program needs	a new bitstream	a flash write; the bitstream stays
Start-up code	`fw/start.S`	`fw/start_spiflash.S`
Speed	4 to 6 cycles an instruction	382 cycles more for each word read from the flash
Simulated	yes: the golden trace	the start-up only (Figure 7.1)

Table 8.1: The two boot modes. `make -C fw` builds either (`TARGET=bram` or `TARGET=flash`), and the README's "Running the demos" gives the steps on the board.

BRAM boot is the simple mode, and the one the regression runs: the program is part of the bitstream and runs at full speed, but it must fit in 6 KB. Five of the six demos do. Flash boot is for a program that does not fit, such as tinyraytracer, 29 KB. Its bitstream is built once, with the default `BOOT_ADDR`, and runs whatever program is written to the code window. A BRAM-boot bitstream sets `BOOT_ADDR` to 0 at synthesis.

8.1 The flash layout

The board's flash is a 32 MB S25FL256S. The core sees its lower 16 MB, laid out as Figure 8.1 shows (`notes.md`, "Flash memory map and boot address").

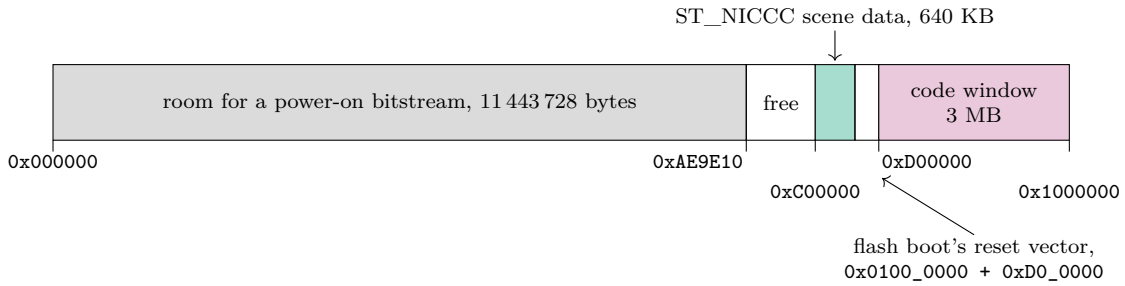


Figure 8.1: The flash layout, to scale: the lower 16 MB of the 32 MB chip, which the core sees at `0x0100_0000`. Offsets are from the start of the chip. On the owner's board, offset 0 holds Digilent's factory demo, whose data reaches up to `0xDEFFFF`, so a write to the code window overwrites part of that data.

Offset 0: room for a bitstream. At power-on, with the mode jumper on QSPI, the FPGA loads its configuration from offset 0. This project never writes there: each of its bitstreams is loaded over JTAG straight into the FPGA and lasts until the power goes, and on the owner's board offset 0 keeps Digilent's factory demo. The layout leaves room there so that a bitstream of this design could be stored to boot at power-on. The room is large because a bitstream configures the whole FPGA, every LUT, flip-flop, routing switch and block RAM, used or not. Its size depends on the device, not on the design: every uncompressed bitstream for the XC7K325T is 11 443 728 bytes, about 10.9 MB, this small SoC's included. The program is not in it: tinyraytracer takes 29 KB of the code window's 3 MB.

The other regions. ST_NICCC's scene data, 640 KB, sits at `0xC00000`, and the code window at `0xD00000`. Each region starts on a 64 KB boundary, because the flash erases in 64 KB sectors, so one region can be rewritten without touching the others.

Why 16 MB. The controller reads with the flash's Read command, opcode `03h`, which carries a 3-byte address, and three bytes reach 2^{24} bytes, 16 MB. That is also why bit 24 selects flash space (Section 4.2).

Writing the code window. A flash-boot write changes only the code window and never offset 0, so the board keeps booting at power-on what it booted before, provided the write keeps the flash's Quad Enable bit (pitfall P8). On the owner's board, a readback of the whole chip after tinyraytracer was written differed from the one before only in the image's own bytes.

8.2 CCLK and STARTUPE2

The flash has four pins. Chip select and the two data lines become ordinary FPGA pins once configuration ends, so they are plain ports in the constraints file. The clock does not: it is CCLK, a dedicated configuration pin, which a design can drive only through the `STARTUPE2` primitive [8]. `SOC` wires `SPIFlash`'s clock to its `USRCCLKO` input and takes back its `EOS` output, end of startup. `IOBus` passes a flash read to the controller only once `EOS` is high (`notes.md`, "SPI flash CCLK access through STARTUPE2").

The hand-over costs the first transaction. Vivado's model of `STARTUPE2` holds CCLK high after end of startup until `USRCCLKO` has risen twice, so the first transaction loses two of its 64 clock edges and the flash misreads its command. In simulation this loses the first fetch of `fw/start_spiflash.S`, and it is the likely reason flash boot needs one press of the reset button after each load on the board (pitfall P5).

8.3 Execute-in-place

In flash boot the code stays in the flash: the core fetches each instruction from there, and `.text` and `.rodata` are never copied. What the program writes must be in the RAM, so `fw/spiflash.ld` puts `.data` and `.bss` there and stores the initial values of `.data` in the flash, after the code. Before `main`, `fw/start_spiflash.S` sets `gp` and `sp`, zeroes `.bss` and copies `.data` into the RAM.

The same copy moves the code that runs most into the RAM, where it runs about a hundred times faster: `libgcc`'s multiply and divide, which every `*`, `/` and `%` in C calls on RV32I, `fw/putchar.S` and `fw/print.c`, which every character passes through, and any function marked `.fastcode`. The code in the flash calls them at their RAM addresses. In mandelbrot's flash build, 1 228 bytes are copied and 1 172 stay in the flash. The copied code shares the 6 KB of RAM with the data and the stack, and nothing checks that the stack stays clear of it (pitfall P9).

9 From C to the board

Figure 9.1 follows one line of the mandelbrot demo, `IO_OUT(IO_LEDS,frame)`, which writes the frame counter to the LEDs, from the C source to the board, and the first instruction of mandelbrot's flash build from its ELF to the flash. The four instructions it becomes are those of Figure 6.3. Every excerpt is cut from the real build by `make flow`, which builds the firmware and a BRAM-boot bitstream in a scratch directory and fails unless each stage holds what the one before made.

From C to machine code. `make -C fw` compiles each C file with `gcc -c` and links the objects, the start-up code and `libgcc` with `ld` into an ELF (`notes.md`, "The compiler and linker flags, one by one"). The store is the 32-bit word `0x0087a223` at address `0x284`, an S-type word whose fields Figure 5.1 decodes. The assembly in the figure is `gcc -S`'s output, written for the reader, and `gcc -c` assembles the same text from a temporary file, so nothing after the compiler reads it. It shows two things the linked code no longer does: `li a5,4194304` is a pseudo-instruction, one `lui` for `IO_BASE`, and `%hi(.LC1)` names a string the linker has not placed yet, so the `lui` reads `0x0` only once the string lands in the first 2 KB.

The firmware image and the block RAMs. The firmware image is the ELF's bytes as text for `$readmemh`: `firmware_words` writes four words a line, each most significant digit first, so the store is word 161, on line 41 (`notes.md`, "Why the ELF-to-hex conversion exists, endianness"). Synthesis turns the RAM into two `RAMB36E1`, each 2K deep and 18 bits wide: `ram_reg_0` holds the low 16 bits of every word and `ram_reg_1` the high 16, so the store's word is split between them, `A223` and `0087`, in `INIT_0A`. These `INIT` values become part of the bitstream, which is why a new program in BRAM boot needs a new bitstream (`notes.md`, "Simulation and hardware after a firmware change").

The bitstream, JTAG and the board. `SOC.bit` begins with a short header, then 11 443 612 bytes of configuration data, 11 443 728 bytes in all, the size of every bitstream for this part, and the FPGA starts reading at the sync word, `0xAA995566` [8]. `board_program.tcl` loads it over JTAG in 6.9 s, and the first byte of frame 0 arrives before Vivado has exited: the FPGA runs the design as soon as configuration ends. The frame takes 1031.3 ms, which Section 1.1 compares with the simulation.

Flash boot. The same C source, linked with `fw/spiflash.ld`, runs from the flash's code window: its first instruction, `lui gp,0x400`, sits at `0x01D0_0000`. `objcopy -O binary` keeps only the bytes, in memory order, so the word `0x004001b7` becomes `b7 01 40 00`, little-endian. `write_cfgmem` wraps the binary in an MCS file, Intel hex for the flash, and `board_flash.tcl` writes those bytes over JTAG and nothing else. The flash-boot bitstream is loaded as on the left, and the core fetches each instruction from the flash. On the board, flash boot ran tinyraytracer (Section 1.1), and mandelbrot's flash build is shown as the same program.

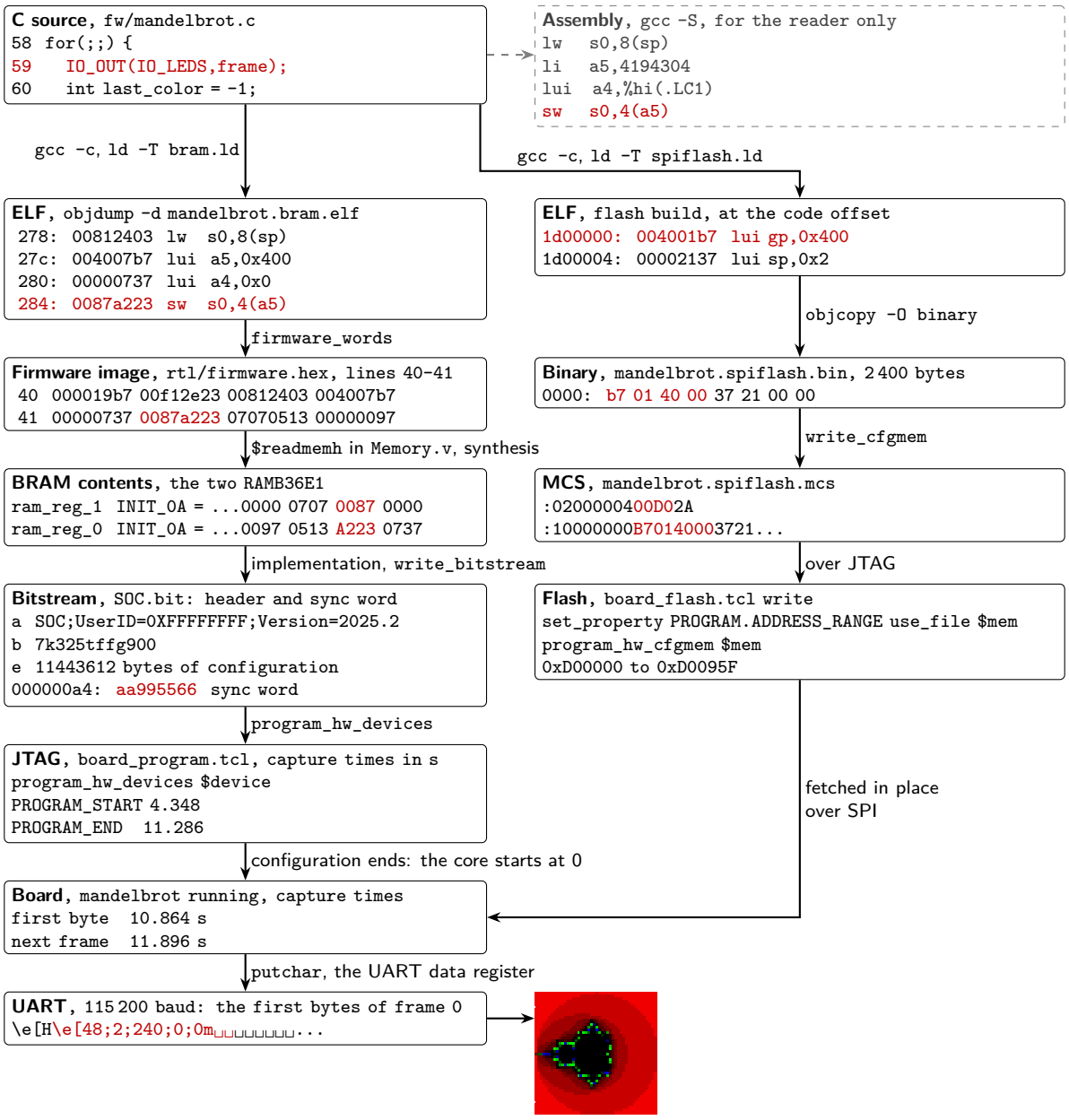


Figure 9.1: The mandelbrot demo from C to the board, in BRAM boot (left) and flash boot (right), with an excerpt of the real build at each stage, generated by `make flow`. In red, what the figure follows: line 59 and the store to the LED register it becomes, at `0x284`, in BRAM boot, and the flash build's first instruction, in flash boot. The assembly, dashed, is made for the reader only. The BRAM halves are spaced 16 bits apart, the most significant first, as the `INIT` string writes them. The UART bytes write `\e` for ESC and ␣ for a space, and the frame they open is drawn beside them. In flash boot the bitstream comes over JTAG as on the left, and the board's times are those of BRAM boot, the only way mandelbrot ran on the board.

10 Verification

The design is checked at three levels: one module against values worked out by hand, the whole SoC against a recorded trace, and the repository against mechanical rules. Each check is a make target, a *gate*, listed in Table 10.1.

Gate	What it runs	Time
`make lint`	the repository's rules (Section 10.4)	seconds
`make unit`	the ALU bench (Section 10.1)	seconds
`make smoke`	`unit`, the firmware check, then the start of the golden trace	25 s
`make check`	`unit`, the firmware check, the whole golden trace, and synthesis	minutes
`make baseline`	synthesis and implementation, with the timing reports of Chapter 11	about a minute

Table 10.1: The main gates, targets of the root `Makefile`, whose header lists them all. The firmware check holds the firmware's copies of the memory map to `rtl/defines.vh`, and rebuilds `rtl/firmware.hex` and fails unless it comes out byte for byte the same. Each gate but `check`, which only runs others, writes a log in `build/logs/` that ends in `PASS` or `FAIL`.

10.1 The ALU unit bench

`sim/ALUBench.v` drives `rtl/ALU.v` directly and checks ten right shifts, `SRL`, `SRA`, `SRLI` and `SRAI`, against results worked out by hand from the RV32I specification [2]. The operands are negative, positive and most-negative, shifted by 0, 4, 9 and 31 places, with the upper bits of the shift amount set, which the ALU must ignore. The operand the instruction does not use is `0xDEADBEEF`, so a wrong operand choice shows.

The bench exists because the golden trace cannot catch a bug that was there when the trace was recorded. `SRA` once shifted logically, and the trace recorded it and agreed with it, until its frame was compared with the C program run natively (pitfall P4). The bench holds the shifts to the specification. The rest of the ALU is held by the trace alone.

10.2 The golden trace

`sim/bench.v` runs `SOC` in BRAM boot and records only what can be seen from outside the chip: each byte on the UART line, decoded from the pin, and each change of the LEDs, with its time. It reads no internal signal, so the RTL can be restructured and the same trace still applies. It writes one line per event and an `END` line:

```
UART 1b
  ⋮
LEDS 1031464038 01
END stop_leds cycles=206292808 uart_bytes=11014 leds=01
```

The run stops when the LEDs become 1, which mandelbrot writes at the start of its second frame, so the trace holds exactly one frame: 206 292 808 cycles, 1.031 s of simulated time and 11 014 UART bytes (`docs/baseline.md`). The `END` line carries the cycle count, so a change that keeps every byte but makes one instruction a cycle slower fails too. The trace's UART bytes are, byte for byte, the frame `fw/mandelbrot.c` prints when compiled natively, and the frame the board sent (Section 1.1).

10.3 The regression

`sim/run_regression.sh` simulates `SOC` and the bench in xsim and passes only when the new trace is byte-identical to the golden one. The full run takes about six minutes. With `--max-cycles` it is the smoke run, about 25 s, whose shorter trace must be the start of the golden one. It runs in its own build directory, since `Memory.v` reads the image by bare name (pitfall P2). `make golden` records a new golden trace, only for an intended change of behaviour, whose commit says why.

The regression has two gaps. It runs one program: mandelbrot uses 24 of the 37 RV32I instructions other than `FENCE`, `ECALL` and `EBREAK`, and never executes `SLL`, `SRA`, `SLT`, `SLTU`, `XOR`, `SLTI`, `SLTIU`, `XORI`, `ORI`, `LB`, `LH`, `LHU` or `SH`. And it runs one boot mode: the bench has no flash model, so flash boot is simulated only by this report's own bench, for Figure 7.1.

10.4 The gates and the lint

`docs/CONTRIBUTING.md`, "Gates and evidence", says which gate a change needs: `make lint` before every commit, `make smoke` after each RTL edit, `make -j2 check` before any push, and `make baseline` when timing must be shown to hold, since `check` stops after synthesis. A commit message cites the log of each gate it ran. Two git hooks enforce this: the pre-commit hook runs `make lint`, and the commit-msg hook refuses a message that cites a missing or failed log, or that claims a pass without citing one. GitHub Actions runs `make lint` again on every push.

`make lint` checks what no reviewer should have to check by eye:

- tool by-products beside the sources (rule 1), the sign of a tool run from the wrong directory (pitfall P2);
- retired paths and words (rule 2), and Markdown lines over 100 characters (rule 3);
- `vivado/rtl_sources.txt` listing exactly the files in `rtl/` (rule 4);
- the naming and banner conventions of `docs/CONTRIBUTING.md` (rules 5, 6, 7 and 9);
- copies equal to their source: every firmware copy of the memory map, and the README's tables, equal to `rtl/defines.vh` (rule 8), and the trace numbers in `docs/baseline.md` equal to the golden trace (rule 10). Each of the two has a self-test that changes every value it checks and expects the rule to fail.

11 Implementation results

Every number in this chapter is from the routed design that `vivado/baseline.tcl` builds, non-project, for the XC7K325T-2FFG900 with the constraints file's 200 MHz clock, in Vivado 2025.2. `make figures` rebuilds that design and fails unless its numbers equal those of `docs/baseline.md`, the reference the gates compare against.

11.1 Utilization

The whole SoC is 984 LUTs and 244 flip-flops, under half a percent of the device, and two of its 445 block RAM tiles. Table 11.1 gives the split per instance.

Instance	LUTs	as RAM	FFs	BRAM tiles
SOC, the whole design	984	44	244	2
`Processor`, the core	941	44	130	0
its own logic: pc, instruction, address adders	89	0	63	0
`Control`, the FSM	58	0	3	0
`ALU`	2	0	0	0
`RegisterFile`	797	44	64	0
`Memory`, the RAM	0	0	0	2
`IOBus`, the LED register	0	0	8	0
`UartTx`	25	0	26	0
`SPIFlash`	19	0	80	0

Table 11.1: Utilization of the routed design, per instance (`report_utilization -hierarchical`). Each row counts where the cells ended up after synthesis, not where their Verilog is. A parent's LUTs can be fewer than its children's sum, since a LUT shared across the hierarchy counts once in the parent. The device holds 203 800 LUTs, 407 600 flip-flops and 445 block RAM tiles, and the design uses no DSP blocks.

The core is nearly all of the logic, and inside it the rows follow the netlist rather than the sources: synthesis optimises across module boundaries, and the ALU's adder, shifter and comparators end up in cells under `cpu/register_file`, leaving 2 LUTs in `ALU`. That is also why the RTL schematic of Figure 6.2 comes from the elaborated design. Of the register file's own LUTs, 44 are distributed RAM: the 32 registers, one copy for each read port. The peripherals are small: `SPIFlash` is mostly flip-flops, its shift registers and counters, `UartTx` is a baud counter and a shift register, `IOBus` is the 8 flip-flops of the LED register, and `Memory` is the two block RAM tiles alone.

11.2 Timing

The one constraint, the 5 ns period of the 200 MHz clock, is met on every one of the 1229 timed endpoints: the worst setup slack (WNS) is 0.219 ns and the worst hold slack 0.113 ns, so the design would close at up to 209.2 MHz. The worst path (Figure 11.1) is the load path.

It starts at `cpu/instr_reg[3]`, an opcode bit of the instruction register, and ends at the data input of the register file's LUT RAM: the write of `rd`. Its first 3 LUTs decode the instruction and choose its immediate, a carry chain of 7 `CARRY4` adds it to `rs1`, the load-store address, as far as bit 24, the bit that selects the flash region, and the last 5 LUTs use that bit to choose the bus's read data, cut out the loaded byte or halfword and pass it to the write port: 15 levels in all. The address is not registered, so the write in `WAIT_DATA` waits on the whole adder again. Of

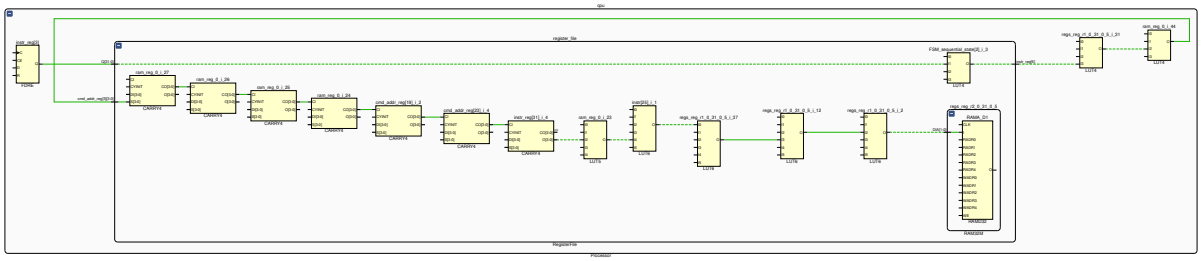
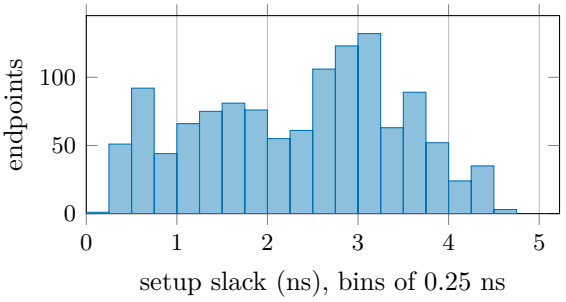
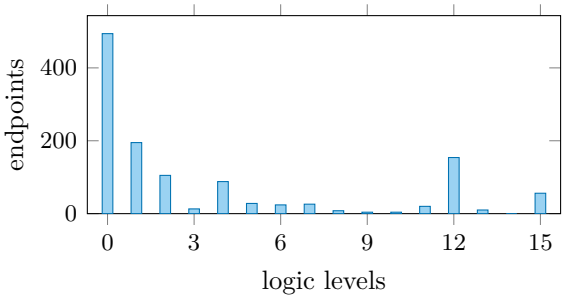


Figure 11.1: The worst setup path in Vivado's schematic, from `instr_reg[3]` at the left to the register file's write data at the right, inside the boxes of `cpu` and `cpu/register_file`: 7 `CARRY4`, 4 `LUT6`, 3 `LUT4`, 1 `LUT5`, between a flip-flop and a LUT RAM. Drawn by `docs/design/scripts/path_schematic.tcl` from the path's pin pairs, so each net shows only its segment on the path.

the path's 4.312 ns, 1.291 ns are in the cells and 3.021 ns, 70 %, in the routing between them. Routing dominates, as it usually does on an FPGA: fewer levels, not faster cells, would shorten the path, and registering the load-store address, or only its region bit, would take the adder off it (pitfall P7). Figure 11.2 shows the margin of the rest of the design: most endpoints have well over a nanosecond to spare, and 494 have no logic at all between their two ends.



(a) Setup slack of each endpoint's worst path. 188 endpoints, 15 %, have less than 1 ns, and the most has 4.516 ns.



(b) Logic levels of the same paths. 56 endpoints sit at the deepest, 15 levels.

Figure 11.2: All 1229 timed endpoints, from `get_timing_paths -nworst 1` on the routed design.

11.3 Power

Vivado estimates 194 mW on the chip (Figure 11.3). 160 mW of it is static: the leakage of the whole Kintex-7, which a configured FPGA draws whatever the design. The design's dynamic power is 35 mW, from 5 to 9 mW each for the block RAMs, the signals, the logic, the I/O and the clock tree. The estimate assumes Vivado's default toggle rates. With a simulation's switching activity it would be more exact, but not different in kind: the core costs a few milliwatts, and the device costs the rest.

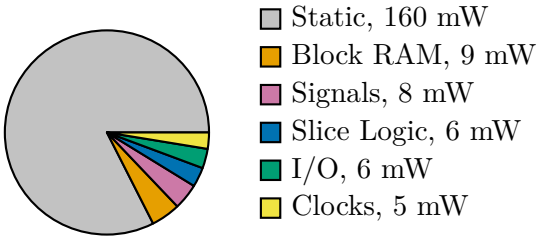
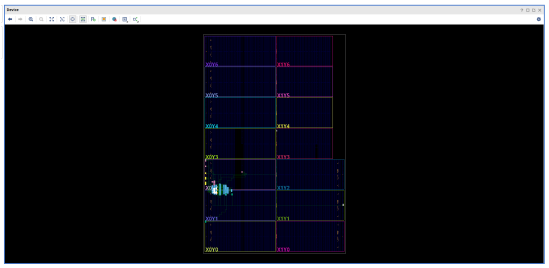


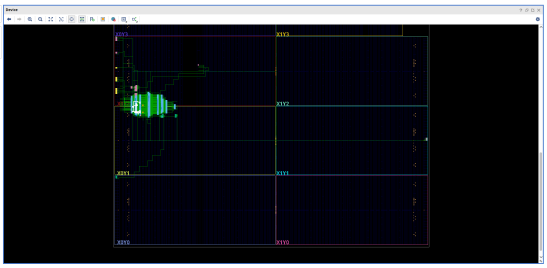
Figure 11.3: On-chip power, 194 mW in all, from `report_power`: a vectorless estimate at Vivado's default toggle rates, with a junction temperature of 25.3 °C. Vivado rounds each figure to the milliwatt, so the parts need not add up to the total.

11.4 The design on the device

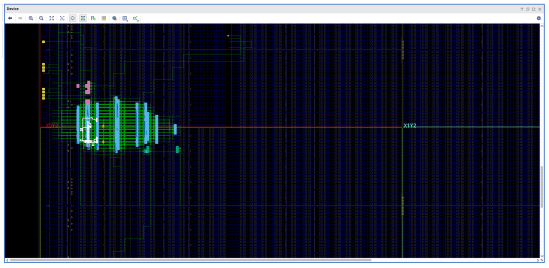
Figure 11.4 is Vivado's Device view of the same routed design, taken by hand in the GUI after `docs/design/scripts/gui_helper.tcl` has coloured the modules and selected the worst path. The whole SoC is a small patch at the left edge of the lower half of the device, and the worst path stays inside it.



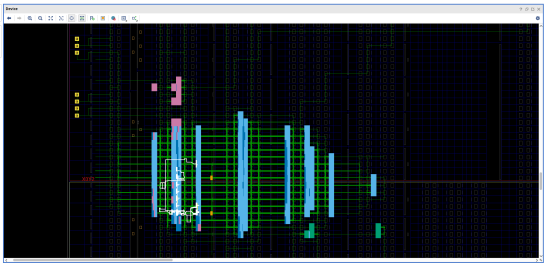
(a) The whole device, its 14 clock regions outlined. The design is the coloured patch at the left edge of the lower half.



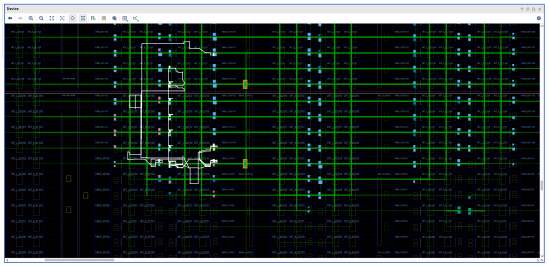
(b) The lower clock regions. Vivado draws the routed nets in green, and some run from the design out to the device's edges.



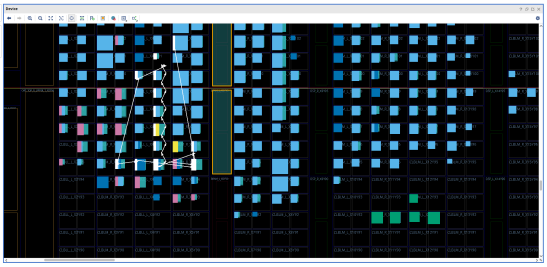
(c) The design: the register file and the rest of the core in the middle, the LED outputs at the left edge, and the worst path in white.



(d) Around the worst path, which ends at a write port of the register file's LUT RAM.



(e) The worst path with Routing Resources on: each net follows the wires the router chose through the interconnect tiles.



(f) Routing Resources off, zoomed further in: each net is a straight line from its driver to its load, over the slices that hold the path's cells.

Figure 11.4: Vivado's Device view of the routed design, zoomed in from the whole device to the worst setup path, `cpu/instr_reg[3]` to the register-file write. The modules are in their colours and the worst path is selected, in white.

12 For the next student

This chapter is for whoever extends the design next: the traps this project fell into, and where to go from here.

12.1 Pitfalls

Each pitfall below cost time once. Each gives the symptom as it showed, the cause, and where the design or the repository now handles it, and the other chapters cite them by number.

P1 The UART pin-direction trap

Symptom. Not one byte reaches the PC and the bridge's activity LED never blinks, while the design looks healthy in simulation.

Cause. The board's net names mix two points of view: TX and RX are the PC's, `_in` and `_out` the FPGA's, so `uart_rx_out` is the FPGA's transmit line. Reading `_in` as "into the PC" swaps the two pins. A testbench that prints what the core handed the UART, instead of decoding the pin, cannot see it.

Handled. The ports of `SOC` keep the board's names and directions, which the constraints file and the testbench bind to, and `sim/bench.v` decodes the `uart_rx_out` pin as a receiver does (`notes.md`, "UART port names and the pin-direction trap").

P2 The firmware image read by bare file name

Symptom. In simulation the program does nothing: the LEDs stay at zero and the RAM reads `x`, with only a warning in xsim's log. After a Vivado run, `.Xil/` and `clockInfo.txt` appear beside the sources.

Cause. `Memory.v` reads `firmware.hex` with `$readmemh` by bare name, relative to the working directory of whichever tool runs, and a missing file leaves the RAM undefined without an error.

Handled. Every script that simulates or synthesises runs in a directory of its own with the image linked or copied in, and lint rule 1 fails on by-products beside the sources (`notes.md`, "Bare file name in $readmemh").

P3 The simulation hang without initial values

Symptom. The simulation runs but the flash controller never starts its clock, and the testbench hangs. The same RTL works on the board.

Cause. A register with no initial value starts as `x` in simulation, and `x + 1` stays `x`: the clock divider of `SPIFlash` never counted. Configuration loads a definite value into every flip-flop, so the silicon never shows it.

Handled. The registers of `rtl/SPIFlash.v` that nothing resets declare initial values: `div` and `sclk` start at 0, and `spi_cs_n` at 1, which alone already sends the controller to its idle branch, where the divider is cleared. The core's registers declare their reset values as initial values. A new counter needs one too (`notes.md`, "Explicit initial values on div and sclk (simulation hang)").

P4 SRA behaving as SRL

Symptom. The golden trace's frame diverged from the C program compiled natively at row 8, pixel 36: `-1544416 » 9` gave `0x007FF437` where `0xFFFFF437` was right.

Cause. In Verilog an expression's operands decide its signedness together. In `funct7[5] ? ($signed(a) >> s) : (a » s)` the unsigned arm makes the whole conditional unsigned, and `>>` then shifts logically.

Handled. Both arms of the shift in `rtl/ALU.v` are signed, `sim/ALUBench.v` checks the four right shifts on negative operands, and the golden trace was captured again. The sign fill cost 54 LUTs (`notes.md`, "SRL versus SRA: both shift arms are signed").

P5 Flash boot loses its first fetch

Symptom. With the flash-boot bitstream loaded over JTAG, no byte arrives on the UART and the LEDs stay dark. One press of the CPU RESET button, and the demo runs from then on.

Cause. Most likely the core's first fetch from the flash is lost straight after configuration. That fetch is `lui gp,0x400`, the first instruction of `fw/start_spiflash.S`, which sets `gp` to `IO_BASE`. Without it the UART writes land in the RAM instead. `STARTUPE2` passes `USRCCLKO` to the CCLK pin only after a few edges, so the first transaction misses clock edges and the flash misreads its command (Section 8.2). And `IOBus` drops, rather than stalls, a fetch requested before end of startup. ST_NICCC shows the same loss in BRAM boot, where the first flash access is a data read: its first frame came out wrong, while the frames after it were drawn from the right data (Section 1.1).

Handled. On the board only: press the reset button once after each load. A candidate fix, not kept in the repository, made the board print straight after the load: `eos` moves into `SPIFlash`, which makes one throwaway read as soon as `eos` rises and holds a read requested before that, busy, until it can start it. It cost about 25 LUTs and 25 flip-flops more and met timing with 0.334 ns of slack.

P6 Reset landing in WAIT_DATA

Symptom. With the reset state changed to `FETCH_INSTR`, the obvious choice, nothing shows in BRAM boot. In flash boot the first fetch after a press of the reset button goes wrong.

Cause. `FETCH_INSTR` asserts the read strobe for as long as it lasts. Held in reset there, with `pc` in flash space, the core would restart the SPI transaction on every cycle, chip select held low: the flash would see a command it never finishes.

Handled. `rtl/Control.v` resets into `WAIT_DATA`, which asserts nothing and falls through to `FETCH_INSTR`. Its write-back is harmless, because the instruction register resets to a no-op whose destination is `x0` (`notes.md`, "Reset lands in WAIT_DATA, not FETCH_INSTR").

P7 The worst path ends in the register-file write

Symptom. A small change on the load path, one more term in the address decode or in the load lanes, can miss the 5 ns of the 200 MHz clock: the worst path has 0.219 ns of slack.

Cause. The load-store address, `rs1 + imm`, is not registered, so every cycle of a load computes it again: 15 levels and 4.312 ns from an opcode bit through the adder, the region decode and the load lanes into the register file's write data (Chapter 11).

Handled. Not yet: a comment at `loadstore_addr` in `rtl/Processor.v` marks it. Register the address in `EXECUTE`, or only its region bit, and the adder leaves the path. `LOAD` and `STORE` follow `EXECUTE`, so the register costs no cycle.

P8 Writing the flash clears the Quad Enable bit

Symptom. After a flash write with "Configuration File Only", the board no longer boots at power-on: DONE off, no error flag, the mode pins right, and every byte of the flash intact.

Cause. Vivado's `program_hw_cfgmem`, with `PROGRAM.SKIP_QE_BIT_ERASE` at its default, clears the S25FL256S's non-volatile Quad Enable bit, which the factory bitstream needs to read the flash four bits wide. A write with an SPIx4 `.prm` does not set it back, and Vivado's Tcl has no command that reads the flash's registers [10].

Handled. `docs/design/scripts/board_flash.tcl write` sets `SKIP_QE_BIT_ERASE` to 1, and a write with it kept the bit. `scripts/flashregs/FlashRegs.v`, loaded over JTAG, prints the flash's registers on the UART and, built with `WRITE_REGS=1`, writes the bit back, which repaired the owner's board (`figures/README.md`, "The Quad Enable bit").

P9 The stack runs into the program

Symptom. A program that has grown wanders off into garbage after a while, or prints nonsense, with no warning from the build.

Cause. In BRAM boot the 6 KB of RAM hold the code, the data and the stack, which grows down from `0x1800` into the same space. In flash boot it shares the RAM with the copied code. Nothing checks how deep the stack goes.

Handled. In part. `firmware_words` refuses an image past `-max_addr`, which `fw/Makefile` sets to the RAM size. Set lower, it reserves room for the stack. The stack itself is not checked (`notes.md`, "firmware_words arguments, -max_addr as a stack guard").

P10 Debug data loaded into the RAM

Symptom. None at run time. The image is larger than its code, though: 852 words, 3 408 bytes, and `objdump -D` shows runs of `.insn` nonsense after `__mulsi3` and `__modsi3`.

Cause. `fw/bram.ld` keeps the catch-all `*(.*)` of [1], which places every section of every input file in the RAM, and Ubuntu's `libgcc.a` carries DWARF debug sections: 1 084 bytes, 18 % of the 6 KB, never fetched but taken from the room the stack grows into.

Handled. Not handled. The committed image is the one this report describes. Adding `*(.debug*)` to `/DISCARD/` gives a 2 324-byte image (`notes.md`, "bram.ld line by line, the /DISCARD/ workaround").

12.2 Extensions, ranked

Ranked by what each gains for the work it takes. None is begun.

1. **Pipelining:** overlap the fetch of one instruction with the execution of the last, to retire close to one instruction a cycle.
2. **The M extension:** multiply and divide in hardware, on the device's DSP blocks, instead of calls into `libgcc`.
3. **UART receive:** a receiver on the unused `uart_tx_in` pin, enough for a serial boot loader.
4. **Interrupts:** the control and status registers of Zicsr and a trap state, with a timer and the UART as the first sources.
5. **DDR3:** the board's 1 GB as main memory, through AMD's Memory Interface Generator and behind a cache.
6. **HDMI:** a video output, from a frame buffer in DDR3.

13 Conclusions

The project set out to run compiled C on a processor of its own, on a real board, and to understand every layer in between. It did. A multi-cycle RV32I core of 984 LUTs, with 6 KB of RAM, three IO registers and a flash controller, runs at the Genesys 2's 200 MHz with 0.219 ns to spare, draws the Mandelbrot set over the UART from the block RAM, and renders a ray-traced picture in place from the flash. Its educational value is the path more than the core: a line of C passes the compiler, the linker, the firmware image, synthesis, the bitstream and the bus before it lights an LED, and building each stage by hand turned layers that are usually black boxes into parts that can be read, changed and checked. The struggles were rarely in the processor. They were the board's net names, a tool's working directory, Verilog's signedness rules, the flash clock's hand-over after configuration and a bit in the flash's configuration register, each a wrong assumption about the tools or the board that looked like a fault in the design. Finding them taught the lesson the project leaves for the next student: a design is as trustworthy as the evidence behind it, here a golden trace that the board matches byte for byte.

Bibliography

[1] B. Levy, *learn-fpga*, "From Blinker to RISC-V", GitHub repository. `https://github.com/BrunoLevy/learn-fpga`

[2] A. Waterman and K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, document version 20191213, RISC-V Foundation.

[3] Digilent, *Genesys 2 Reference Manual*. `https://digilent.com/reference/programmable-logic/genesys-2/reference-manual`

[4] Digilent, *Genesys 2 Schematic*, revision H.1, 22 pages, 2015, in the Genesys 2 Resource Center. `https://digilent.com/reference/programmable-logic/genesys-2/start`

[5] D. Sokolov, *tinyraytracer*, "Understandable RayTracing in 256 lines of bare C++", GitHub repository. `https://github.com/ssloy/tinyraytracer`

[6] Oxygene, *ST-NICCC 2000*, Atari ST demo, and its polygon stream as published for the ST-NICCC competition. `http://arsantica-online.com/st-niccc-competition/`

[7] FTDI, *FT232R USB UART IC Datasheet*.

[8] AMD Xilinx, *7 Series FPGAs Configuration User Guide*, UG470.

[9] AMD Xilinx, *7 Series FPGAs SelectIO Resources User Guide*, UG471.

[10] AMD Xilinx, *Vivado Design Suite Tcl Command Reference Guide*, UG835.